TOBB UNIVERSITY OF ECONOMICS AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

GUIDE

(Guided User Itinerary & Destination Explorer)

Final Report

QUAD-CORE Team

Ebrar Sude Doğan
Erdem Baran
Kayrahan Toprak Tosun
Tuna Kodal

Contents

1 Introduction **4**
1.1 Abbreviations 4

2 Background and Related Work **4**
2.1 Commercial Travel Planning and Mapping Systems 4
2.2 Alternative Systems and Approaches 5
2.3 Algorithmic and Technical Foundations 5

3 System Overview **7**
3.1 Functional Mapping to Requirements 8
3.2 Key Design Decisions 9
3.3 Dataset Construction 10
3.3.1 Aggregate POI Data from Multiple Sources 10
3.3.2 Merge and Deduplicate POIs 10
3.3.3 Clean and Filter the Dataset 11
3.3.4 Assign Hierarchical Categories 12
3.3.5 Add Planning Attributes 13
3.3.6 Integrate Multimedia Content 14
3.3.7 Prepare the Dataset for System Usage 14
3.4 Dataset Analysis 15
3.4.1 Data Structure Analysis 15
3.4.2 Dataset Analysis 16

4 Algorithmic Design **18**
4.1 Candidate Route Generation 19
4.2 Estimated Visit Duration 20
4.3 Scoring and Ranking Mechanism 21
4.4 Itinerary Generation Algorithm 23
4.5 Route Evaluation 23

5 System Architecture **23**
5.1 Presentation Layer 24
5.2 Backend API Gateway Layer 24
5.3 Backend Services Layer 24
5.4 Integration Layer 25
5.5 Data Layer 26

6 Implementation and Technologies **26**
6.1 Backend Implementation and Technologies 26
6.2 Frontend Implementation and Technologies 27
6.3 External Services Implementation and Technologies 27
6.4 Testing Implementation and Technologies 28
6.5 Use of Modern and Emerging Technologies 28

7 Testing and Evaluation **29**
7.1 Testing Methodology 29

7.1.1 Unit Testing . . . 29
7.1.2 Integration Testing . . . 30
7.1.3 System Testing . . . 30
7.1.4 Performance Testing . . . 31
7.1.5 User Testing . . . 31
7.2 Test Results . . . 32
7.2.1 Quantitative Test Results . . . 32
7.2.2 User Evaluation Results . . . 32
7.3 Evaluation of Results . . . 33
7.3.1 Heuristic Evaluation Limitations . . . 33
7.3.2 User Testing Constraints . . . 34
7.3.3 Dataset and Media Evaluation Gaps . . . 34
7.3.4 External Dependency Testing Limitations . . . 34

8 Observed Issues and Resolutions 35
8.1 Constraint Handling and Edge Case Issues . . . 35
8.2 Algorithmic Tuning and Scoring Imbalance . . . 35
8.3 Routing Integration Issues (OSRM) . . . 35
8.4 Data Consistency and Mapping Issues . . . 35
8.5 Dataset Quality and External Data Issues . . . 36
8.6 User Interface and Responsiveness Issues . . . 36

9 Impact of the Engineering Solution 36
9.1 Economic Impact . . . 36
9.2 Societal Impact . . . 37
9.3 Environmental Impact . . . 37
9.4 Legal and Ethical Impact . . . 37
9.5 Security Impact . . . 38

10 Contemporary Issues 38
10.1 Increasing Consumption and Experience-Oriented Travel . . . 38
10.2 Platform Neutrality and Bias-Free Recommendations . . . 38
10.3 Acceleration and Demand for Instant Solutions . . . 38
10.4 Information Overload and Limited Attention Span . . . 39
10.5 Data Quality and Content Reliability . . . 39
10.6 AI and Automation in Decision-Making . . . 39
10.7 Privacy and Data Minimization . . . 39
10.8 User-Centered Design Considerations . . . 39

11 Discussion 39
11.1 Constraints . . . 40
11.1.1 Algorithmic and Performance Constraints . . . 40
11.1.2 Geographic Scope Constraints . . . 40
11.1.3 Data and Real-Time Information Constraints . . . 40
11.1.4 Dataset Coverage and Content Constraints . . . 40
11.1.5 External Dependency Constraints . . . 41
11.1.6 User Model and Interaction Constraints . . . 41
11.1.7 Development and Resource Constraints . . . 41

11.2 Design Trade-offs 41
11.3 Relation to Future Work 41
11.4 Engineering Contributions 42

12 Conclusion and Future Work 42
12.1 Future Work 42
12.1.1 Planning and Itinerary Enhancements 42
12.1.2 Data and Infrastructure Improvements 43
12.1.3 User Experience and Interaction 43
12.1.4 Personalization and Social Features 43

References 43

13 Appendices 46

A Example POI Record 46

B Category-Based Visit Duration Values 47

C Itinerary Generation Algorithm 47

D Frontend Structure 49

E Backend Structure 49

1 Introduction

Travel planning is inherently a complex and fragmented process that requires users to combine information from multiple independent sources. Travelers typically rely on a mixture of navigation services, review platforms, travel blogs, and personal scheduling tools to construct an itinerary. This fragmented workflow increases cognitive load, requires repetitive manual effort, and often results in inefficient or inconsistent travel plans. In particular, integrating spatial constraints, time limitations, and personal preferences into a coherent multi-day itinerary remains a challenging task for users.

To address these challenges, this project introduces **GUIDE** (**G**uided **U**ser **I**tinerary and **D**estination **E**xplorer), an integrated travel planning system that transforms user-defined constraints into structured and route-aware itineraries. GUIDE consolidates itinerary generation, route optimization, interactive map visualization, and destination content delivery into a single platform. By combining user preferences such as trip duration, category interests, and distance constraints with algorithmic route generation, the system produces optimized multi-day travel plans. Additionally, GUIDE enhances the exploration experience by providing curated textual, visual, and audio content for each point of interest, as well as enabling dynamic replanning based on user modifications.

This report presents the design, implementation, and evaluation of the GUIDE system. It covers the dataset construction process, algorithmic methods used for itinerary generation, system architecture, implementation details, and testing results. The report also discusses the broader impact of the system and outlines potential directions for future improvements. To maintain clarity and avoid unnecessary repetition, detailed content from previously submitted documents, including the proposal, project specification, analysis, high-level design, low-level design, and test plan reports, is not repeated; instead, these documents are referenced where appropriate [27, 28, 25, 26, 29, 30].

1.1 Abbreviations

The following abbreviations are used throughout this report.

GUIDE : Guided User Itinerary and Destination Explorer
POI : Point of Interest
OSM : OpenStreetMap
OSRM : Open Source Routing Machine
API : Application Programming Interface
JSON : JavaScript Object Notation
TTS : Text-to-Speech
LLM : Large Language Model
MCTS : Monte Carlo Tree Search
TSP : Traveling Salesman Problem
TTDP : Tourist Trip Design Problem

2 Background and Related Work

This section presents the background resources and related work relevant to the GUIDE system. Existing approaches are examined under three categories: widely used commercial systems, alternative systems and approaches, and academic studies. This categorization allows for a clearer understanding of how current solutions address different aspects of travel planning, while also providing the conceptual and technical background that informs the design decisions and algorithmic choices adopted in the GUIDE system.

2.1 Commercial Travel Planning and Mapping Systems

A number of widely used commercial systems provide partial solutions to travel planning by focusing on either routing, point-of-interest discovery, or itinerary organization. However, these systems typically lack an integrated approach that combines all aspects into a unified workflow.

Mapping and navigation platforms such as Google Maps offer highly accurate routing and location-based services, enabling users to discover and navigate between points of interest efficiently [22]. However, these systems do not provide automated itinerary generation; instead, users are required to manually select and organize POIs into a coherent travel plan.

Similarly, routing engines such as the Open Source Routing Machine (OSRM) provide efficient and scalable route computation based on geographic data [34]. While highly effective for pathfinding, these systems require predefined input locations and do not include mechanisms for POI selection or itinerary construction.

Travel planning platforms such as TripAdvisor allow users to organize their travel plans by saving and arranging selected destinations [42]. However, the system primarily relies on manual POI selection and does not generate structured itineraries automatically.

Wanderlog provides a more structured itinerary planning interface, combining map visualization with day-by-day trip organization [45]. Nevertheless, it still depends on user-driven POI selection, and certain core functionalities, such as route visualization on the map, are restricted behind a premium model.

Roadtrippers focuses on road trip planning and route-based exploration [39]. However, its geographic coverage is primarily limited to certain regions and does not support destinations for Türkiye, resulting in a scope that differs from the focus of this project.

Sygic Travel Maps offers itinerary planning and travel guide features through a dedicated application [41]. While it provides structured planning tools, the requirement of installing a separate application introduces additional friction for users, particularly for infrequent travel scenarios.

Overall, existing systems address individual components of travel planning but lack an integrated approach. Consequently, users are required to combine multiple tools to construct complete itineraries. None of these systems explicitly provides a unified, end-to-end solution that combines automated itinerary generation with comprehensive coverage tailored to Türkiye while minimizing user effort.

2.2 Alternative Systems and Approaches

Alternative systems and approaches to travel planning adopt different design paradigms, ranging from infrastructure-level routing engines to user-facing AI-driven planning tools. While some systems focus on accurate and efficient route computation, others aim to improve user interaction through more flexible and intuitive interfaces, such as natural language-based planning.

OpenTripPlanner (OTP) is an open-source multimodal trip planning system designed primarily for public transportation routing [35]. It supports combinations of transit, walking, and cycling, and incorporates real-time updates such as delays and service disruptions. However, OTP focuses on transportation networks and route computation rather than tourism-oriented itinerary generation, and does not include mechanisms for selecting or ranking points of interest.

GraphHopper provides routing and route optimization services with support for multiple transportation modes and constraints such as time windows and vehicle capacities [24]. While it offers efficient pathfinding and optimization capabilities, it operates primarily as an API-driven routing service and does not address POI discovery or automated itinerary construction from a user perspective.

Recent approaches such as Layla explore the use of natural language processing for travel planning [33]. These systems allow users to generate travel plans through textual prompts, representing a more interactive and flexible interface. However, they often rely on subscription-based models and lack transparent or structured control over route generation and constraint handling, limiting their applicability for deterministic itinerary planning.

Overall, these systems highlight different directions in travel planning, including infrastructure-level routing, optimization services, and AI-driven interaction. However, they do not offer a unified solution that combines POI selection, route construction, and constraint-aware itinerary generation within a single integrated open-source workflow.

2.3 Algorithmic and Technical Foundations

This section outlines the algorithmic and technical foundations that informed the design of

the GUIDE system. Rather than following a single predefined methodology, the system was developed through an iterative engineering process, drawing on established techniques in routing, heuristic optimization, data processing, and natural language generation. Relevant concepts and methods from the literature are referenced where applicable to position the implemented approach within existing research directions.

The design of the GUIDE system is conceptually aligned with established research in itinerary planning, particularly within the framework of the Tourist Trip Design Problem (TTDP) and the orienteering problem family [19, 43]. In these formulations, the objective is to construct feasible travel plans by selecting and ordering a subset of points of interest (POIs) under constraints such as limited time, travel distance, and user preferences [19]. GUIDE follows a similar paradigm by generating daily itineraries that balance POI relevance with feasibility, reflecting the core idea of maximizing user utility under practical travel constraints [43].

However, enabling such itinerary construction introduces a complex underlying optimization problem. In particular, the task can be related to the Traveling Salesman Problem (TSP), a well-known combinatorial optimization problem that seeks the shortest possible tour visiting each location exactly once [32]. As the number of locations increases, the solution space grows combinatorially, making exact optimization computationally expensive and often impractical for real-time, interactive applications.

For this reason, many itinerary planning approaches in the literature adopt heuristic and stochastic strategies rather than attempting to compute globally optimal solutions [43]. These methods generate multiple candidate solutions and evaluate them using scoring or utility functions, allowing efficient exploration of large search spaces while maintaining practical feasibility. The GUIDE system follows this paradigm by prioritizing high-quality feasible solutions over strict optimality.

In particular, Monte Carlo-based methods provide a sampling-driven approach to decision making in complex search spaces. Such methods iteratively explore alternative solution configurations through stochastic sampling and evaluate them based on predefined criteria, enabling the system to focus on promising regions of the search space without requiring full enumeration [15]. Originally developed and widely applied in domains such as game playing, Monte Carlo Tree Search (MCTS) and related techniques have demonstrated strong performance in problems where exhaustive optimization is infeasible.

Inspired by these principles, the itinerary generation approach in GUIDE adopts a heuristic, sampling-based strategy. The system generates multiple candidate routes through stochastic exploration and selects among them using a composite scoring function. This approach aligns with established heuristic optimization strategies, where the goal is to efficiently obtain high-quality solutions in large combinatorial spaces rather than guaranteeing global optimality.

In addition to route construction, GUIDE incorporates key elements emphasized in TTDP-based systems, including the treatment of POI visit duration as a planning-relevant parameter, support for multi-day itinerary generation [19]. These features enable the system to reflect both structural constraints and user preferences within a unified planning framework.

Beyond routing and planning, GUIDE also reflects the broader paradigm of mobile tourism recommender systems by integrating itinerary generation with rich POI presentation. Modern travel systems increasingly combine recommendation, route construction, and content delivery to enhance user experience [18]. GUIDE follows this approach by coupling structured route generation with multilingual descriptions and multimedia content, supporting both planning and exploration within a single workflow.

Finally, the system leverages recent advances in natural language processing for content-related tasks. Large language models (LLMs) have become effective tools for text generation, paraphrasing, and multilingual translation [14, 44]. Their ability to produce coherent and context-aware content makes them suitable for applica-

tions requiring scalable and consistent textual descriptions.
In particular, lightweight models such as Claude Haiku, developed by Anthropic, enable these capabilities to be integrated into practical systems via API-based access [12]. Such models support efficient content transformation and multilingual processing, reducing the need for manual curation. The adoption of LLM-based processing in GUIDE therefore reflects the integration of modern natural language technologies into the travel planning pipeline, where such models are used to refine, transform, and extend textual content associated with POIs.

3 System Overview

The GUIDE system operates as an end-to-end workflow that transforms user-defined travel preferences into a structured, multi-day itinerary. At a high level, the system integrates input collection, feasibility validation, route generation, and itinerary construction into a unified pipeline.
The overall workflow of the system is illustrated in Figure 1
The process begins with the user providing high-level inputs, including the destination city, desired trip duration, daily distance constraints, and preferred categories. These inputs are validated to ensure feasibility with respect to the available POI dataset. In cases where the requested configuration is not feasible (e.g., insufficient POIs for the selected categories and duration), the system provides feedback by suggesting an alternative number of days or allowing the user to revise their category selections. Following this feedback, the user may choose to proceed with the suggested configuration or revise the input parameters. If the suggested trip duration is accepted, the system continues the planning process using the current selection and generates an itinerary accordingly. Otherwise, the user can modify preferences such as category selections, after which the validation step is repeated.
Once the inputs are finalized, the system proceeds to itinerary generation. The planning process follows an iterative, day-by-day strategy. For each

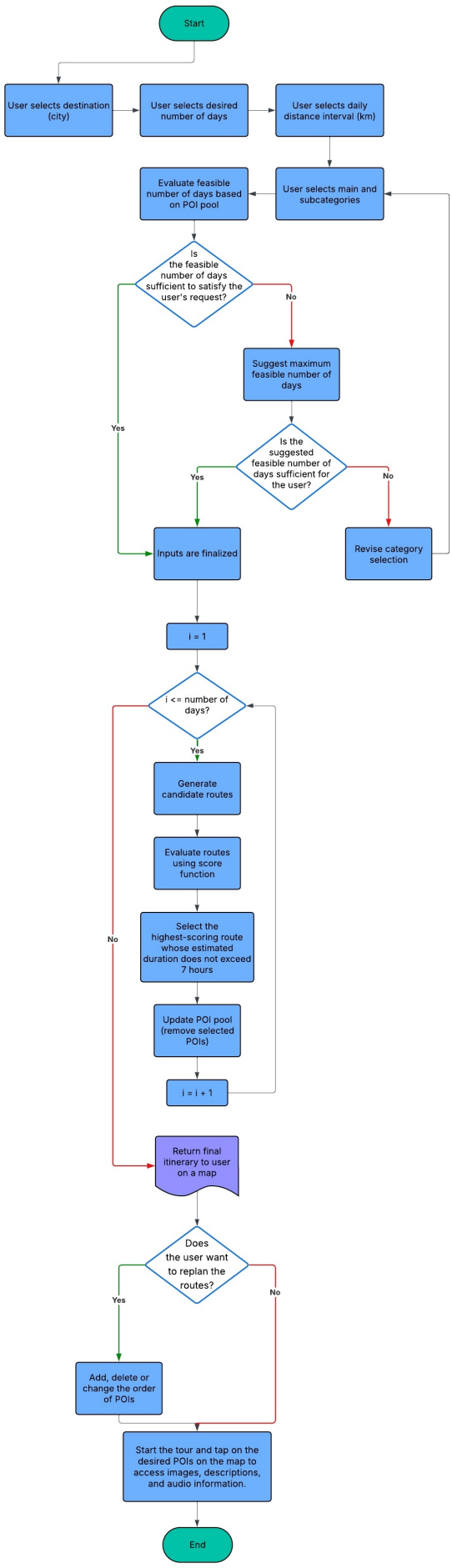

Figure 1: High-level workflow of the GUIDE system.

day, a set of candidate routes is generated from the current POI pool. These routes are evaluated using a heuristic scoring function that considers factors such as distance constraints, category relevance, and route structure. The highest-scoring route is selected for that day, and the included POIs are removed from the pool to prevent repetition across the itinerary.

This iterative process continues until all days are planned. The final output is a complete itinerary consisting of ordered POIs grouped by day, along with their associated route structure. The generated itinerary is then returned to the user for visualization and interaction.

In addition to the initial planning phase, the system supports a flexible replanning workflow. After the itinerary is generated, users can modify the plan by adding or removing points of interest, enabling dynamic adjustments to the route structure. Unlike the initial planning stage, this phase does not enforce strict feasibility constraints; instead, it prioritizes user control and flexibility, allowing the itinerary to evolve according to individual preferences.

Furthermore, the system incorporates lightweight time estimation mechanisms to improve the practical usability of generated itineraries. Estimated visit durations for POIs and approximate travel times between locations are considered to provide a more realistic representation of daily plans. These estimations support better user understanding of route feasibility without introducing heavy optimization overhead.

This section provides a high-level view of the system, while detailed explanations of the dataset, algorithmic design, and system architecture are presented in the following sections.

3.1 Functional Mapping to Requirements

This section establishes a functional mapping between system requirements and their implementation in the GUIDE system. The mapping is presented at a functional level to complement the architectural description and to avoid repetition of the full Requirement Traceability Matrix (RTM), which is provided in the Test Plan document [30]. These requirements are formally defined and categorized in the Project Specification Report, which serves as the primary reference for system functionality and constraint definitions [28].

User Input and Constraint Definition (FR-H01, FR-L01) User inputs such as city, trip duration, distance constraints, and category preferences are collected through the Presentation Layer and transmitted to the backend via RESTful API calls. These inputs are validated at the API boundary using structured request schemas, ensuring completeness, type correctness, and feasibility before entering the planning pipeline. The validated inputs are then transformed into structured parameters used by the itinerary generation services, ensuring that all planning operations are directly driven by user-defined constraints.

Preference-Aware Itinerary Generation (FR-H02, FR-L02, FR-L06) The system generates multi-day itineraries within the Backend Services Layer using a Monte Carlo-based candidate generation strategy combined with a multi-criteria scoring mechanism. Candidate routes are evaluated based on normalized factors such as POI popularity, distance compliance, category diversity, and POI count. The ranking and selection process is implemented through dedicated planning and scoring components, ensuring that the final itinerary reflects user preferences while remaining computationally feasible.

Interactive Replanning Mechanism (FR-H03, FR-L03) The replanning functionality is implemented as a stateless backend service that processes modified POI sequences received from the frontend. User-driven changes, including POI insertion, removal, or reordering, are propagated to the planning pipeline, where only affected route segments are recomputed through the routing integration layer. This selective recomputation strategy preserves unchanged segments while maintaining overall itinerary consistency.

Multimedia Content Delivery (FR-H04, FR-L04) Each POI is enriched with structured textual descriptions, images, and location data retrieved from the Data Layer through repository-style access components. The separation of core POI metadata from content and media assets enables efficient query patterns, allowing lightweight planning operations to remain independent from content retrieval processes used for user interaction.

Multilingual Text-to-Speech Support (FR-H05, FR-L05) Text-to-speech content is generated during an offline data preparation pipeline using third-party TTS services and stored as pre-computed audio assets. At runtime, the system retrieves language-specific audio files from the Data Layer, avoiding real-time TTS computation and ensuring low-latency playback. This design separates content generation from runtime delivery and improves overall system responsiveness.

Route Visualization and Map Interaction (FR-H06, FR-L07, EIR-H01, EIR-L01, EIR-L02, EIR-L03) The Presentation Layer integrates interactive map components that render route geometries obtained from the Integration Layer. The frontend consumes structured route responses and visualizes them using map-based UI modules, supporting zooming, panning, and POI interaction. This ensures consistent visualization across different devices and screen sizes while maintaining a clear separation between rendering logic and backend computation.

Visit Duration Estimation Integration (FR-H07) POI visit durations are estimated within the Backend Services Layer using a category-based heuristic model augmented with popularity signals such as ratings and review counts. These estimates are incorporated into route feasibility checks during candidate evaluation, ensuring that generated itineraries satisfy daily time constraints.

Robust Error Handling and Validation (FR-H08, FR-L08, FR-L09) Input validation and error handling are performed at the API boundary and service layers. Invalid or infeasible requests are intercepted before entering the planning pipeline, and errors are communicated through structured API responses with explicit status indicators. This approach prevents unhandled exceptions from propagating to the client and ensures predictable system behavior under both normal and edge-case scenarios.

Data Source Transparency (FR-H09, FR-L10, FR-L11) The system explicitly communicates the origin of routing and POI data through the user interface and response metadata. Routing results are derived from OpenStreetMap-based services via OSRM, while POI content is obtained from curated public datasets. This improves transparency and aligns with system-level requirements for user awareness and data provenance.

Route Generation Performance Compliance (PR-H01) The system satisfies performance requirements through the use of heuristic sampling, constrained candidate evaluation, and selective routing integration. Computationally expensive exact optimization methods are avoided, and the candidate space is bounded to ensure that itinerary generation completes within the target response time under typical usage conditions.

3.2 Key Design Decisions

Heuristic-Based Planning: Since route generation must complete within a short time frame to support real-time user interaction, exact optimization approaches were avoided in favor of a heuristic strategy that remains tractable under combinatorial complexity.

Monte Carlo-Based Candidate Generation: As an exhaustive enumeration of the solution space is infeasible in real time, candidate routes are generated through stochastic sampling rather than full enumeration. This allows the system to explore a large search space efficiently while maintaining interactive performance.

Composite Scoring Function: Since a single

evaluation criterion is insufficient to capture the diversity of user preferences, distance compliance, popularity, category diversity, spatial coherence, and route size are combined into a unified scoring function to enable balanced and preference-aware candidate selection.

Dataset-Centric Design: To minimize runtime dependency on external services, the system is built around a preprocessed POI dataset enriched with semantic and planning attributes, enabling efficient filtering and candidate generation without repeated external calls.

Separation of Planning and Routing: To avoid managing two distinct computational loads simultaneously and to preserve the independent testability of each module, itinerary construction and road-level routing are decoupled. OSRM is invoked only for the final selected route, reducing unnecessary routing overhead during candidate generation [34].

Multi-Stage Pipeline: Since a single-stage generation process complicates quality control and allows errors to propagate unchecked, the system follows a staged pipeline consisting of candidate generation, filtering, scoring, and selection to incrementally refine solutions.

User-Controlled Replanning: Recognizing that user preferences may evolve after the initial itinerary is generated, the replanning phase operates with relaxed constraints, prioritizing user autonomy over strict feasibility enforcement.

3.3 Dataset Construction

A central component of the GUIDE system is the construction of a curated Point of Interest (POI) dataset tailored to tourism-oriented itinerary generation in Türkiye. Since no single source provides sufficiently complete, clean, and structured coverage for this purpose, the dataset was built through a multi-stage pipeline that combines heterogeneous data collection, merging, cleaning, categorization, enrichment, and final preparation for system integration. Throughout this process, particular emphasis was placed on preserving tourism relevance, improving geographic and semantic consistency, and maintaining a stable schema suitable for downstream planning and recommendation tasks. The overall construction workflow is illustrated in Figure 2.

3.3.1 Aggregate POI Data from Multiple Sources

The initial version of the GUIDE dataset was constructed by aggregating POI records from multiple heterogeneous sources, each contributing different types of information. OpenStreetMap (OSM) served as the primary backbone for geographic data, providing coordinates and baseline POI entries through the Overpass API [7]. This base dataset was expanded using Wikipedia and Wikidata, which were utilized to identify culturally significant locations and to retrieve descriptive content when available [11, 10]. In addition, Google Places was employed to enrich POIs with popularity-related attributes such as rating, review counts, and available media content [2].

Despite the coverage provided by these sources, the collected data remained incomplete and inconsistent, particularly for less prominent or region-specific locations. To address this limitation, supplementary web scraping was performed across a wide range of publicly available resources. Approximately 150 distinct sources were explored, including municipal websites, local tourism portals, cultural heritage listings, and region-specific informational pages, such as official institutional platforms (e.g., [3, 5, 9]). These sources were used primarily for validation, completion of missing POIs, and refinement of existing records.

A comprehensive list of these external sources is provided separately to avoid cluttering the main text. The full collection of reference links can be accessed via this link.

3.3.2 Merge and Deduplicate POIs

Following the aggregation of POIs from multiple heterogeneous sources, a dedicated merging and deduplication step was required to construct a unified and consistent dataset. Since different data sources often refer to the same physical location using slightly different names, coordinates, or category labels, direct matching based on exact

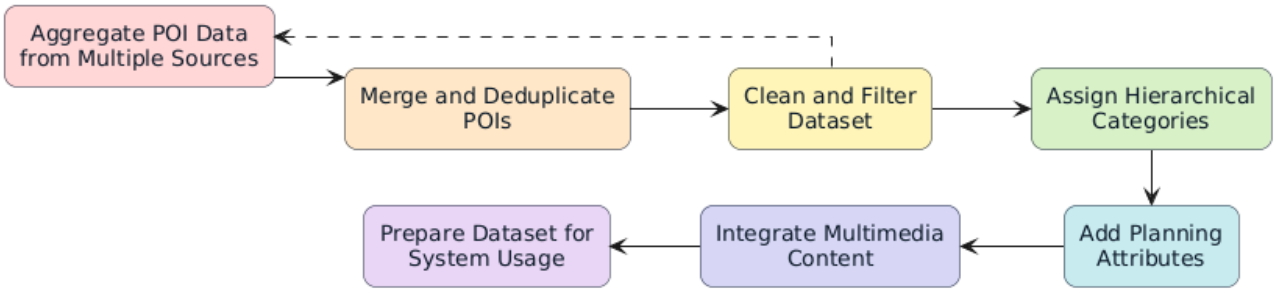

Figure 2: Overview of the GUIDE POI dataset construction pipeline

equality was not sufficient.
To address this, a heuristic-based matching strategy was employed, combining both spatial and textual similarity. Each POI was represented using a composite key constructed from its geographic coordinates and name. Latitude and longitude values were rounded to a fixed precision, effectively grouping locations within a limited spatial tolerance, while a truncated version of the POI name was used to capture its semantic identity. This approach enabled the identification of candidate duplicates that refer to the same real-world entity but differ slightly across sources.
For each group of candidate matches, records were merged into a single unified POI entry. Rather than selecting a single source, attributes from different records were combined in a controlled manner. Descriptive fields were prioritized based on source reliability and content completeness, while categorical and metadata fields were aggregated to preserve the diversity of available information. In cases where multiple textual descriptions were available, preference was given to descriptions obtained from more structured and descriptive sources, such as Wikipedia or those providing more complete contextual information, avoiding redundancy across merged entries [11].
Conflicts between sources were resolved using a predefined source prioritization strategy established during dataset construction. In this strategy, more structured and descriptive sources, such as Wikipedia, were generally preferred over less detailed entries, while numerical attributes obtained from APIs were incorporated when available. This approach ensures that each merged POI reflects a consistent and practically reliable combination of the available data.
In addition, source information was preserved for each POI during the merging process, allowing the origin of each record to be tracked across different data sources. This enables easier validation, supports incremental dataset updates, and improves the overall transparency of the integration process. As a result, the merging and deduplication stage transforms a fragmented and partially overlapping collection of POIs into a coherent and non-redundant dataset, providing a reliable foundation for subsequent processing steps.

3.3.3 Clean and Filter the Dataset

Following the merging stage, the resulting dataset required extensive cleaning and validation due to the heterogeneous and user-generated nature of the collected data. Although the aggregation process provided broad coverage, the raw dataset contained various inconsistencies, noisy entries, and semantically invalid POIs that were not suitable for tourism-oriented itinerary generation.
The initial cleaning phase focused on textual normalization and structural consistency. POI names and descriptions were manually reviewed to correct encoding issues, inconsistent naming conventions, and non-standard representations, including entries containing foreign alphabets or malformed text. In addition, duplicate naming patterns, irrelevant labels, and structurally inconsistent records were identified and corrected to ensure a uniform representation across the dataset.
A significant challenge arose from the use of openly editable sources such as Wikipedia. While these sources provided valuable coverage, they also introduced a considerable amount of unre-

liable or low-quality content. In particular, certain entries corresponded to user-generated or unverified locations that lacked cultural, historical, or touristic relevance. Examples include locally named but non-recognized landmarks, informal place definitions, or geographically trivial locations that were incorrectly labeled as POIs. Such entries were systematically identified and removed based on their lack of relevance to the intended use of the system.

In addition to textual validation, a comprehensive manual filtering process was applied to ensure the semantic quality of the dataset. Each province was reviewed individually, and POIs were evaluated based on their touristic significance, distinctiveness, and suitability for itinerary generation. This process involved the removal of redundant or overly granular entries (e.g., multiple sub-components of the same location), as well as entries representing minor or non-distinct geographic features that do not contribute meaningfully to user itineraries.

Multimedia data also required careful curation. Although images were initially retrieved through automated APIs, they frequently contained irrelevant or misleading content, such as personal photographs, incorrectly labeled views, or images unrelated to the actual POI. To address this, all images were manually inspected, and only representative and contextually accurate visuals were retained for each POI. This step ensured that the visual content aligned with the semantic identity of the location.

Overall, the cleaning stage combined automated filtering techniques with extensive manual validation to transform a noisy and inconsistently structured dataset into a high-quality, tourism-oriented POI collection. This process required substantial effort but was essential to ensure the reliability and usability of the dataset for downstream itinerary generation and user-facing applications.

3.3.4 Assign Hierarchical Categories

The assignment of hierarchical categories was introduced after observing that category information obtained directly from external sources was insufficient in terms of consistency, granularity, and completeness. Although multiple sources, including OpenStreetMap and Wikipedia, were explored to extract categorical metadata, the resulting labels were often sparse, overly generic, or structurally incompatible with the requirements of itinerary generation [7, 11]. As a result, a dedicated categorization process was developed to construct a controlled and consistent category hierarchy tailored to the needs of the GUIDE system.

The hierarchical categorization stage was applied to make the dataset usable for preference-based filtering and itinerary generation. The objective of this stage was not only to assign a single label to each POI, but to represent each location through a structured category hierarchy that could capture both broad thematic identity and finer-grained semantic distinctions.

The initial categorization process was carried out using a rule-based labeling pipeline built on keyword sets and regular-expression matching. Candidate category signals were searched across the textual fields available for each POI, and pseudo-labels were assigned accordingly. These pseudo-labeled records were then grouped and reviewed under a hierarchical structure consisting of main categories and subcategories. This design enabled the dataset to retain semantic richness while remaining suitable for downstream filtering and scoring operations.

Because many POIs matched multiple semantic patterns, an additional consolidation step was required to control excessive labeling. During this stage, the hierarchy was explicitly constrained to at most two main categories and at most four subcategories per POI. Records exceeding these limits were manually revised, and lower-priority labels were removed to preserve consistency and interpretability across the dataset. This decision was particularly important for locations with mixed cultural, architectural, religious, or natural characteristics, where unconstrained labeling would otherwise produce overly noisy or unbalanced records.

The categorization stage also included manual handling of records that could not be confidently

labeled through the rule-based pipeline. 146 POIs remained unlabeled after the automated categorization phase. These records were reviewed individually; 22 were manually incorporated into the final categorized dataset, while the remaining entries were excluded after manual evaluation due to their lack of relevance or insufficient significance for itinerary generation. In this way, category assignment was treated not as a purely automatic annotation task, but as a controlled semantic curation process combining rule-based labeling with manual validation.

As a result, this stage introduced a structured semantic layer over the previously uncategorized POI collection, enabling category-aware querying, filtering, and ranking. The resulting hierarchy forms a critical bridge between raw destination data and the planning logic of the GUIDE system.

3.3.5 Add Planning Attributes

Following semantic categorization, the dataset was extended with planning-oriented attributes to support the itinerary generation process. While the dataset primarily represents POIs, these additional attributes were introduced to guide and constrain the planning space, ensuring that user inputs remain within feasible and computationally manageable bounds. In this sense, the dataset was augmented not only to describe locations but also to provide lightweight structural information that improves the efficiency and stability of downstream planning operations.

The max_days attribute was defined to limit the maximum trip duration supported for each city. This value is not an intrinsic property of individual POIs; instead, it reflects a city-level constraint derived from the distribution and density of available POIs.

This constraint was determined empirically by observing how many days can be populated with meaningful and well-balanced routes given the available POIs. Rather than maximizing the possible trip length, the goal is to ensure that each day contains a sufficient number of relevant POIs. The mapping is implemented as a threshold-based function over the total POI count of a city. By restricting unsupported durations at the interface level, this attribute prevents low-quality planning scenarios and simplifies subsequent validation and computation steps.

In addition to duration constraints, distance-based control parameters were introduced to bound route generation. For each city, minimum and maximum route distances were estimated using a Monte Carlo-style sampling procedure. A large set of candidate routes was generated, with the number of samples scaled proportionally to the number of available POIs (POI count $\times$ 5000). Each sampled route contained between 4 and 9 POIs, reflecting the practical structure of daily itineraries considered in the system.

The resulting route distances were analyzed statistically to derive stable operational limits. The upper bound was determined using the 95th percentile of the sampled distances to mitigate the effect of extreme outliers, while the lower bound was computed as the average of the 25 shortest sampled routes, capturing realistic minimum travel scenarios. This choice was based on the empirical observation that meaningful minimum routes are typically represented within the lowest fraction of the sampled distribution.

Since the route samples used in the Monte Carlo process are generated based on geometric (straight-line) distances between POIs, the initially computed route lengths reflect Euclidean approximations rather than actual travel distances. However, real-world travel distances are obtained through road networks and are therefore typically longer than straight-line estimates.

To account for this discrepancy, all distances were scaled using an empirically determined factor of 1.65. This factor was derived from experiments conducted across different regions of Türkiye, where straight-line distances between POI pairs were compared with their corresponding road network distances obtained via OSRM. The observed ratios between geometric and routing-based distances were analyzed over a set of samples, resulting in an approximate scaling factor of 1.65 for typical road conditions in Türkiye.

Finally, both bounds were discretized into 5-kilometer intervals by rounding the lower bound downward and the upper bound upward, resulting in stable and interpretable control values for the application.
These planning attributes do not directly determine the generated itineraries; instead, they act as guiding constraints that reduce the search space and improve the responsiveness of the planning pipeline. These constraints are primarily enforced at the interface level by defining the allowable input ranges presented to the user, rather than through strict validation during the planning phase. By embedding such control parameters into the dataset, the system achieves a balance between flexibility and feasibility, enabling efficient and reliable itinerary generation.

3.3.6 Integrate Multimedia Content

Multimedia integration was carried out in two main parts: visual content collection and audio content generation. For visual assets, Google Places API was used to retrieve candidate photographs for each POI [2]. Since these images may also include user-uploaded and contextually irrelevant content, a manual cleaning process was applied after collection. For each POI, a limited number of representative images were selected, unsuitable or misleading photographs were removed, and in cases where the initial results were not satisfactory, the retrieval and filtering process was repeated.
For audio content, the process built upon the textual descriptions collected during earlier dataset construction stages. These texts, obtained and completed using sources such as Wikipedia, OpenStreetMap-related data, and Google-based metadata, were often not directly suitable for speech output because they contained long, irregular, or stylistically inconsistent sentences [11, 7, 2]. To improve readability and listening quality, the texts were revised using the Claude Haiku model [12]. In this step, the Turkish and English descriptions were paraphrased into clearer and more concise forms, with particular attention to sentence flow, coherence, and suitability for text-to-speech usage. Since German descriptions were not available in the original collection, they were generated by translating the refined English texts using the same model.
After textual refinement, audio guides were generated in Turkish, English, and German through Google's text-to-speech API [23]. In this way, the multimedia integration stage combined automated content retrieval and generation with substantial manual review, resulting in a cleaner and more consistent set of images and audio assets for user-facing presentation.

3.3.7 Prepare the Dataset for System Usage

After the preceding stages, the dataset was largely complete in terms of content; however, it was not yet directly usable within the GUIDE system. Several inconsistencies and missing values remained, particularly in location assignments and API-derived metadata fields. Therefore, a final preparation stage was required to ensure that all records were complete, consistent, and compatible with the planning pipeline.
The first issue addressed in this stage was incomplete or inconsistent city assignment. Since POIs were collected from multiple heterogeneous sources, city labels were not always reliable or present. To resolve this, a two-step approach was applied. Existing city fields were first normalized to a consistent naming format. For POIs with missing or ambiguous city information, reverse geocoding was performed using their geographic coordinates to assign a valid province-level location. Additionally, coordinate-based validation was applied to ensure that all POIs fall within the geographical boundaries of Türkiye. Records that did not satisfy this constraint were removed, as they were considered outside the scope of the project.
The second issue involved missing popularity-related attributes obtained from external APIs, particularly google_rating, and google_reviews_total. These attributes play a critical role in the system, as they are directly used in the computation of POI popularity and influence the scoring and ranking stages of itinerary generation. There-

fore, handling missing values required careful consideration.
Since these values were not available for all POIs, a controlled completion strategy was adopted. Missing values were filled using city-level distributions in order to preserve local characteristics. During exploratory analysis, it was observed that widely recognized and frequently visited POIs almost always had complete metadata, while missing values were predominantly associated with less prominent or less-documented locations.
Based on this observation, a conservative imputation approach was employed. For each city, the lower decile of available values was used as a reference, and missing entries were assigned values from this range. This design choice reflects the assumption that POIs with missing popularity signals are more likely to belong to the lower end of the popularity spectrum. As a result, the imputation strategy prevents artificially inflating the importance of such POIs in subsequent ranking and scoring stages, while still allowing them to remain part of the candidate set.
Formally, let S_c denote the set of POIs in city c for which a given attribute is available. The missing values were estimated from the lower segment of this distribution:

$$x_{\text{missing}} \sim \text{LowPercentile}(S_c)$$

where $\text{LowPercentile}(S_c)$ represents the lower 10% of observed values within the city. This conservative strategy maintains relative consistency while avoiding distortion of popularity-based features.
In addition to metadata completion, limitations of external APIs were addressed through a fallback retrieval strategy. In some cases, direct queries to Google Places did not return sufficient information for a given POI, particularly for less prominent or sparsely documented locations [2]. To mitigate this issue, additional queries based on spatial proximity (nearby search) were performed using the POI coordinates. This allowed the system to recover missing contextual information and associate relevant metadata with otherwise incomplete entries. As a result, POIs that would have been excluded due to insufficient API responses could be retained within the dataset.
Finally, the dataset was transformed into a system-ready format suitable for direct integration into the GUIDE pipeline. This included standardizing field names, ensuring consistent null-value representations, and organizing media references into a uniform structure compatible with storage and retrieval mechanisms. Additionally, all records were validated to conform to a consistent schema, enabling seamless usage across backend services and frontend components without requiring additional runtime preprocessing. This step ensures that the dataset can be directly consumed by the itinerary generation and exploration modules of the system.

3.4 Dataset Analysis

This section analyzes the structure and composition of the dataset.

3.4.1 Data Structure Analysis

Each POI in the GUIDE dataset is represented as a structured record that integrates geographical, categorical, descriptive, and multimedia information within a unified schema. This structure was designed to support both itinerary generation and user-facing exploration.
A complete example of a POI record is provided in Appendix A (Hagia Sophia). The main attributes of the dataset are described below.
name : Name of the POI.
lat, lon : Geographic coordinates used for routing and spatial analysis.
city : Assigned province-level location in Türkiye, used for filtering and planning.
main_1, main_2 : Primary semantic categories representing high-level thematic groups (maximum two).
sub_1 – sub_4 : Fine-grained subcategories providing additional semantic detail (maximum four).
categories : Raw category labels obtained during data aggregation, preserved for reference.
medium_description_* : Multilingual textual descriptions (TR, EN, DE), derived from

medium-length content extracted from source platforms (e.g., Wikipedia), where multiple description lengths (short, medium, long) are available[1].
audio_* : Audio guide files derived from textual descriptions for multimodal interaction[1].
photo_urls, photo_files : External and locally stored image references associated with the POI.
google_rating, google_reviews_total : Popularity-related attributes obtained from Google Places, used in scoring and ranking.
This schema provides a balance between descriptive richness and structural consistency, enabling efficient processing within the planning pipeline while maintaining a high-quality user experience. It should be noted that the attributes described above correspond to the application-level representation of the dataset, which is optimized for itinerary generation and user interaction. The underlying database schema contains additional structural components that are introduced in the System Architecture section.

3.4.2 Dataset Analysis

Dataset Overview The final GUIDE dataset consists of a total of 2,337 POIs covering all 81 provinces of Türkiye. This ensures nationwide geographical coverage and allows the system to generate itineraries across diverse regions and tourism profiles.
In addition to structural completeness, the dataset incorporates rich multimedia content. A total of 5,767 photographs are associated with POIs, with 2,091 out of 2,337 POIs containing at least one image. This corresponds to a high visual coverage ratio, ensuring that the majority of locations are supported with representative imagery. On average, each POI is associated with 2.47 images.
Audio content is provided for all POIs through multilingual text-to-speech generation. A total of 7,872 audio files are available, corresponding to three language variants (Turkish, English, and German) per POI. This results in an average of 3.00 audio files per POI, confirming complete audio coverage across the dataset.

[1]The * denotes language variants: **tr**, **en**, **de**.

The textual descriptions used for content generation exhibit a consistent level of detail. The average length of English descriptions is 613 characters, indicating a balanced level of informativeness without excessive verbosity. Descriptions in other languages (Turkish and German) follow a similar length distribution due to the standardized generation and translation process applied across all entries. This consistency supports both readability and effective audio rendering.
A detailed analysis of audio durations was conducted over the entire dataset. The overall duration statistics are as follows: the average audio length is 39.99 seconds, with a minimum of 23.86 seconds and a maximum of 79.15 seconds. These values indicate a controlled and relatively narrow distribution, reflecting the standardized content generation pipeline.
When analyzed by language, audio durations exhibit slight variation. Turkish audio files are shorter on average (35.35 seconds), English recordings occupy a mid-range (39.32 seconds), and German audio files are longer (45.30 seconds). This variation is primarily attributed to linguistic differences in sentence structure and word length.

Geographical Distribution The spatial distribution of POIs across Türkiye is illustrated in Figure 3. Each point on the map represents a POI included in the dataset. The distribution demonstrates that all regions of the country are covered, with higher densities observed in major tourism hubs such as coastal regions and historically significant urban centers.
This distribution reflects both the availability of cultural and natural assets and the emphasis placed on tourism relevance during dataset construction. The presence of POIs across all provinces ensures that the system can support itinerary generation at a national scale, while the varying densities capture realistic differences in tourism activity.

City-Level Distribution The distribution of POIs across cities reveals significant variation in terms of density and coverage. The city with

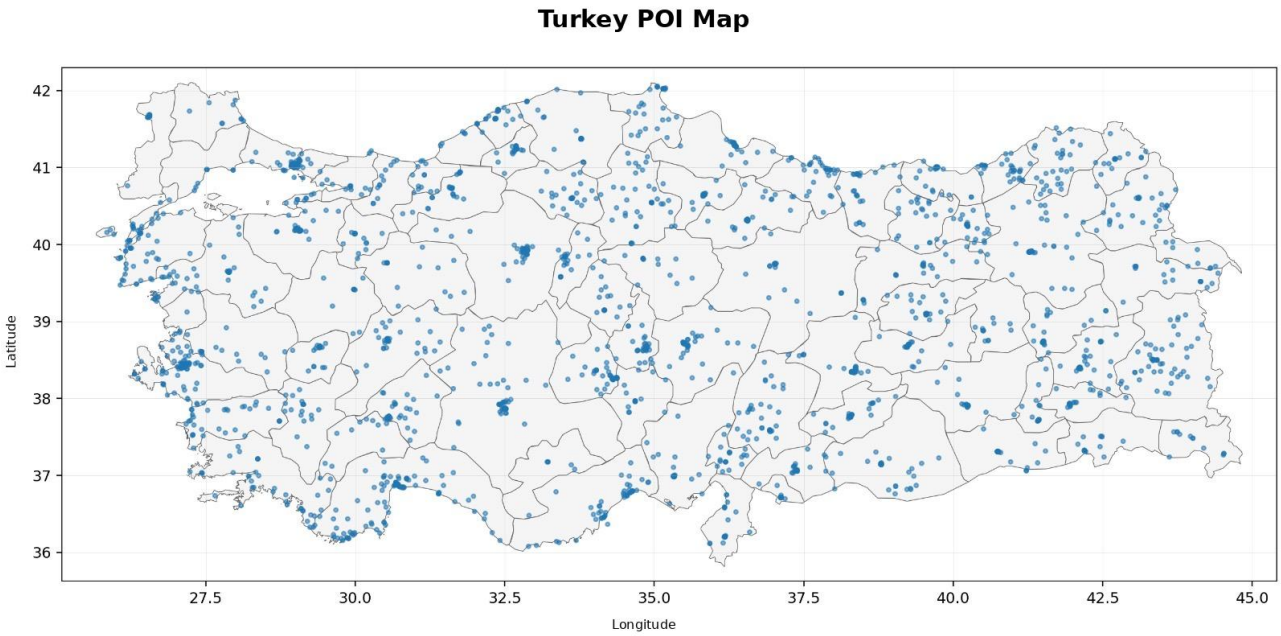

Figure 3: Geographical distribution of POIs across Türkiye.

the highest number of POIs is Antalya, with 107 entries, followed by İzmir (105), İstanbul (98), Ankara (71), and Konya (64). These cities correspond to major tourism and cultural centers, confirming that the dataset aligns with real-world travel demand.

At the lower end of the distribution, cities such as Karaman (6), Iğdır (8), Tekirdağ (8), Kırklareli (9), and Zonguldak (9) contain fewer POIs. This reflects either lower tourism density or limited availability of documented locations in these regions.

This variation in POI density across cities directly affects the itinerary generation process, as regions with higher POI availability allow for more diverse and flexible route construction, while lower-density regions impose tighter constraints on feasible itineraries.

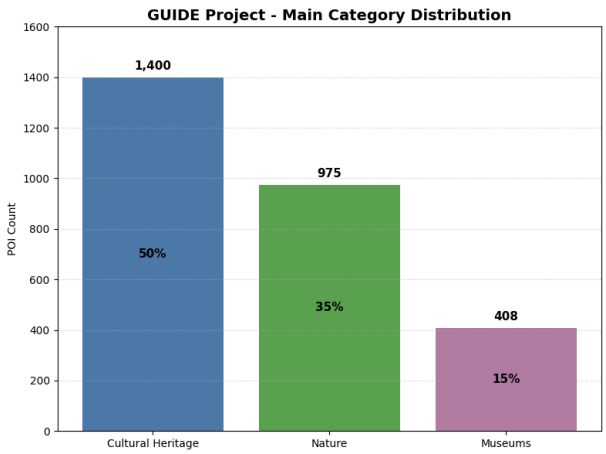

Figure 4: Distribution of POIs across main categories.

Category Distribution The distribution of POIs across main categories is shown in Figure 4. The dataset is dominated by Cultural Heritage locations, which account for 1,400 POIs (50.3%), followed by Nature-related locations with 975 POIs (35.0%), and Museums with 408 POIs (14.7%).

This distribution reflects the tourism profile of Türkiye, where historical landmarks, natural attractions, and cultural sites form the core of visitor interest. This distribution aligns with the primary focus of the dataset construction process, where culturally and historically significant lo-

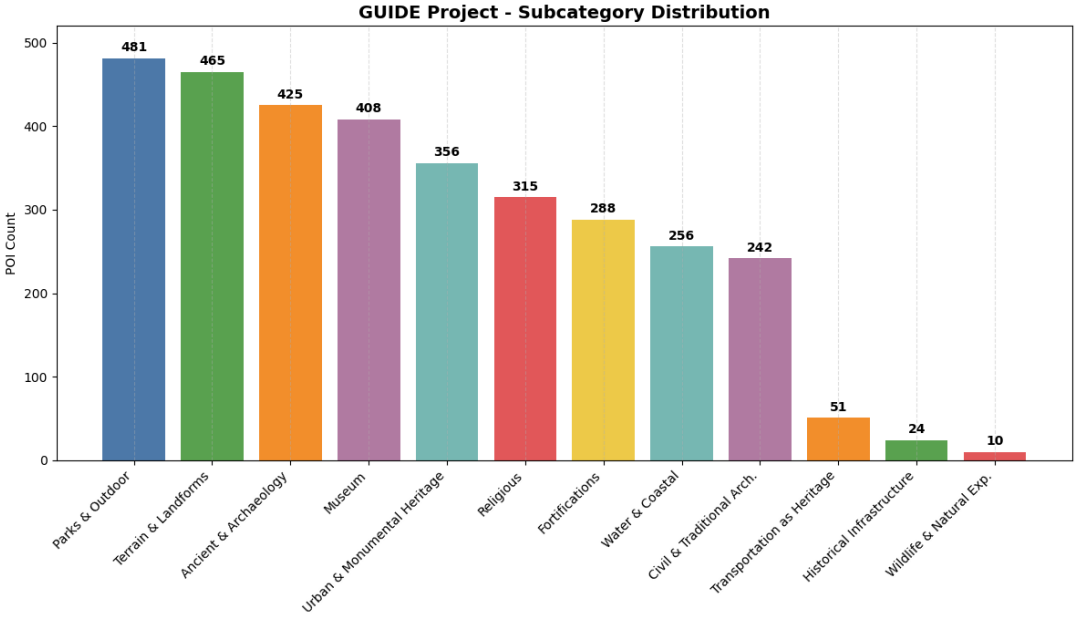

Figure 5: Distribution of POIs across subcategories.

cations were intentionally prioritized to support tourism-oriented itinerary generation.

A more detailed breakdown is provided through subcategory distribution, as illustrated in Figure 5. The most frequent subcategories are Parks & Outdoor (481), Terrain & Landforms (465), Ancient & Archaeology (425), Museum (408), and Urban & Monumental Heritage (356). This distribution indicates that the dataset is strongly oriented toward outdoor, natural, and historically significant locations, reflecting the dominant tourism characteristics of Türkiye.

The hierarchical relationship between main categories and subcategories is visualized in Figure 6. This structure enables the system to support both high-level filtering and fine-grained categorization, which are essential for user preference modeling and itinerary generation.

Overall, the dataset exhibits a well-balanced distribution across geographical regions, categories, and content modalities. The combination of structured metadata, rich multimedia content, and consistent statistical properties makes it suitable for both algorithmic processing and user-facing applications.

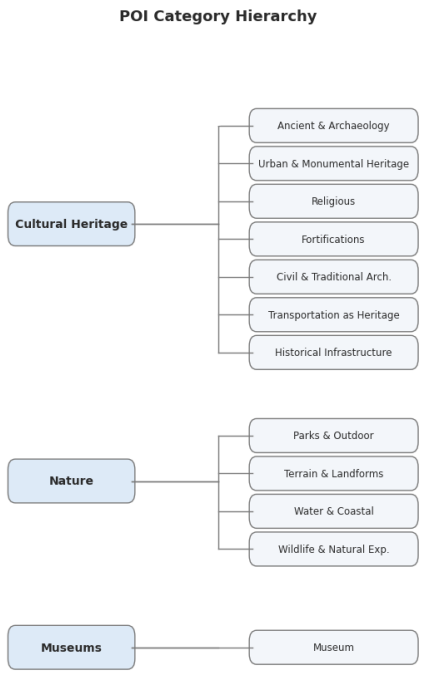

Figure 6: Hierarchical structure of categories.

4 Algorithmic Design

This section presents the main algorithmic components of the GUIDE system within the practical constraints of real-world itinerary generation. The problem addressed by the system is inherently complex, as it requires constructing multi-day travel plans from heterogeneous and unevenly distributed POI datasets across 81 differ-

ent cities. The variability in POI density, category composition, and data completeness makes it difficult to apply exact optimization methods in a consistent and scalable manner.

While the itinerary construction task can be related to combinatorial optimization problems such as the Traveling Salesman Problem (TSP), solving such formulations exactly is computationally impractical in an interactive setting where user-defined constraints must be handled dynamically [32]. Moreover, the problem extends beyond pure distance minimization, requiring the integration of semantic preferences, category diversity, popularity signals, and temporal feasibility.

For these reasons, the system adopts a constraint-aware, heuristic design. Instead of searching for globally optimal solutions, the approach focuses on generating high-quality feasible routes under practical limitations. This is achieved through a combination of structured filtering, heuristic scoring, lightweight duration estimation, and Monte Carlo-style sampling, allowing the system to operate efficiently while maintaining meaningful and user-aligned itineraries.

4.1 Candidate Route Generation

The itinerary generation process begins with the construction of a city-specific POI pool. Given the user's selected city and category preferences, the system retrieves all relevant POIs from the dataset and filters them according to the requested semantic constraints. Since the dataset contains both main categories and finer-grained subcategories, this filtering step allows the system to represent user intent at a sufficiently specific level while keeping the search space manageable.

Based on this filtered pool, the system generates candidate daily routes using a Monte Carlo-style sampling strategy. For each day, a fixed number of candidate routes ($N = 500$) is generated through random sampling. If no feasible route is identified after evaluation, the generation process is repeated up to three times using newly sampled candidates.

Each candidate route contains between 4 and 9 POIs, reflecting the practical structure of a single-day itinerary.

For each candidate route, the order of POIs is determined using a nearest-neighbor heuristic. The starting point is selected as the POI farthest from the centroid of the candidate set, and subsequent POIs are chosen iteratively based on minimum geographic distance (Haversine). This provides a lightweight approximation of route structure prior to routing.

Instead of evaluating all generated candidates equally, a top-k filtering strategy is applied. Only the highest-scoring candidates ($k = 10$) are further evaluated against feasibility constraints such as duration and distance bounds. This reduces computational overhead while preserving solution quality.

Among the top-k candidates, the total visit duration of each route is computed by summing the estimated visit durations of its POIs. Routes whose total visit duration exceeds the predefined daily feasibility limit ($7hours$) are filtered out at this stage.

In addition to the duration check, only routes whose total distance falls within a tolerance band of 75% to 125% of the target distance are accepted as feasible.

The remaining routes constitute a set of feasible candidates for the given day. The algorithm proceeds by evaluating these complete route candidates in the next stage using a composite scoring mechanism. This design enables the system to assess each route holistically, considering factors such as distance consistency, overall popularity, and categorical composition. As a result, route selection is performed based on the overall quality of the itinerary, rather than on isolated local decisions.

This design choice is also motivated by a practical travel consideration. When constructing daily itineraries, selecting a single highly attractive but geographically distant location often comes at the cost of visiting multiple reasonably attractive locations within a compact area. In many cases, such choices lead to inefficient routes with excessive travel time and reduced overall exploration. To address this trade-off, the system favors route

configurations that balance attractiveness with spatial coherence. In addition to distance-related factors, the scoring mechanism also incorporates attributes such as popularity, category diversity, and route structure. Since these factors can only be meaningfully evaluated at the route level, the algorithm operates by generating a pool of candidate routes and selecting among them, rather than making decisions at a more granular level. This design allows multiple criteria to be considered jointly, resulting in more balanced and practically useful itineraries.

4.2 Estimated Visit Duration

In itinerary generation, the system requires an estimate of both the time spent at POIs and the travel time between them. The visit duration component is used during candidate filtering to enforce the daily feasibility limit, while travel time, obtained later from the routing engine, contributes to the final user-facing itinerary timing. Travel times between POIs are obtained through the OSRM, which provides distance and duration estimates based on real road network data, and are therefore treated as a reliable external input to the planning process [34].

In contrast, the time spent at each POI is not consistently available from external sources (e.g. Google Places). Since no single reliable and complete source exists for visit duration information, this component is estimated within the system. Additionally, visit duration estimation is a core functional requirement of the system, as it directly influences itinerary feasibility and time-constrained route planning. These estimates are used both during itinerary generation, particularly in filtering routes that exceed the daily duration constraint, and in presenting users with a realistic expectation of daily plans.

The model combines category-level prior knowledge with popularity-related signals. According to the data preparation notes, the estimation process consists of four stages:

(i) category-based base duration assignment,
(ii) review-count adjustment,
(iii) rating-based adjustment, and
(iv) final clipping and rounding.

First, each POI is assigned a category-based initial duration. Since no standardized and complete source exists for visit durations, representative duration values were defined for each category based on empirical observations and typical visitor behavior. These category-level duration assignments are provided in Appendix B.

While these values serve as a general baseline, certain POIs may exhibit significantly different real-world visit characteristics compared to their assigned category. In such cases, manual adjustments were applied to better reflect practical visit durations. For example, although Anıtkabir is categorized under monuments, its actual visit duration is substantially longer than typical entries in this category and was therefore manually corrected.

Since some POIs belong to multiple categories, a single duration value is derived by combining category-level information. Let max_duration denote the duration associated with the dominant (i.e., highest-impact) category of the POI, and let avg_duration denote the average duration across all assigned categories. The initial duration is then computed as:

$$\begin{aligned}\text{category_duration} = 0.70 \times \text{max_duration} \\ + 0.30 \times \text{avg_duration}\end{aligned} \tag{1}$$

This formulation ensures that the primary category remains the dominant factor, while still incorporating secondary category information in a controlled manner.

Next, the duration is adjusted using popularity-related signals derived from user interaction data. A review-based multiplier m_{review} is applied to capture the observation that widely visited locations tend to require longer engagement times, whereas less-known locations are typically explored more quickly. Rather than using raw review counts directly, which could lead to disproportionate influence, review values are discretized into predefined intervals. Each interval is associated with a bounded multiplier within the range $[0.90, 1.10]$. POIs with relatively low review counts are assigned multipliers closer to

0.90, while highly reviewed POIs receive values approaching 1.10. This piecewise mapping ensures that popularity contributes to duration estimation in a controlled manner without introducing excessive variance.

Similarly, rating values are transformed into a bounded multiplier $m_{\text{rating}} \in [0.90, 1.10]$. Higher-rated POIs are assigned slightly larger multipliers to reflect increased user engagement, while lower-rated POIs receive smaller adjustments. The mapping is designed to be smooth and limited in effect, ensuring that rating differences refine the estimation without dominating it.

Both m_{review} and m_{rating} are intentionally bounded within a narrow range to act as secondary modifiers. This ensures that category-based duration remains the dominant factor in the estimation, while popularity signals provide controlled refinement.

These multipliers act as moderate scaling factors that adjust the category-based duration according to the relative popularity and perceived importance of the POI, while preserving the stability of the overall estimation process.

An intermediate duration value is computed by incorporating popularity-based adjustments:

$$\begin{aligned}\text{adjusted_duration} = \text{category_duration} \\ \times\, m_{\text{review}} \\ \times\, m_{\text{rating}}\end{aligned} \tag{2}$$

The final visit duration is obtained by applying normalization and bounding operations to this value:

$$\begin{aligned}\text{duration} = \max\Big(15,\ \min\Big(150, \\ \text{round}\Big(\frac{\text{adjusted_duration}}{5}\Big) \times 5\Big)\Big)\end{aligned} \tag{3}$$

The rounding operation ensures that durations are expressed in consistent 5-minute intervals, while the bounding operation constrains the results between 15 and 150 minutes to prevent unrealistically short or excessively long visit times.

This estimation mechanism provides a lightweight yet structured approach for incorporating temporal realism into itinerary generation, while remaining robust to missing or incomplete external data sources.

4.3 Scoring and Ranking Mechanism

Following the candidate route generation stage, the system must evaluate and compare multiple feasible route alternatives in order to select the most suitable itinerary for each day. Since each candidate route represents a complete daily plan, the selection process requires a holistic evaluation rather than a sequence of local decisions.

To address this, GUIDE employs a composite scoring mechanism that integrates multiple aspects of route quality, including popularity, spatial coherence, category diversity, and route size. The objective is not to optimize a single metric, but to balance several competing factors that jointly determine the practical usefulness and attractiveness of an itinerary.

At the POI level, one of the primary signals is the popularity score, computed using Google-derived metadata. The popularity of a POI is defined as:

$$\begin{aligned}\text{Popularity}(p) = {} & google_rating(p) \\ & \times \log\big(1 + google_reviews_total(p)\big)\end{aligned} \tag{4}$$

This formulation combines rating quality and review volume into a single metric. While rating reflects users' subjective evaluation of a POI, it is not sufficient on its own, since a high score based on a very small number of reviews does not provide reliable evidence of overall quality. Therefore, the number of reviews is incorporated to capture the level of consensus and reliability of the rating.

The logarithmic transformation ensures that the review count contributes to the score without disproportionately dominating it, thereby preserving a balance between widely known landmarks and moderately popular locations. The additive constant inside the logarithm prevents undefined values in cases where the review count is zero and ensures numerical stability.

As these attributes directly influence ranking, missing values were handled conservatively during dataset preparation to avoid artificial score inflation as discussed before.

While popularity captures the attractiveness of individual POIs, route-level evaluation also re-

quires assessing how well the overall itinerary aligns with user-defined spatial constraints. In particular, the total travel distance plays a critical role in determining the practical feasibility and usability of a route.

Since invoking the routing engine for every sampled candidate would be computationally prohibitive, total route distance during scoring is approximated by applying a constant detour factor of 1.65 to the cumulative Haversine distance between consecutive POIs. This factor empirically approximates the typical ratio between straight-line and road-network distances in urban areas. Exact road-network distances are subsequently obtained from OSRM only for the selected route.

To model this aspect, the distance component evaluates how closely the total route distance aligns with the target distance using a Gaussian function:

$$f_{\text{distance}}(r) = \exp\left(-\frac{(d(r) - d_{\text{target}})^2}{2(\sigma)^2}\right) \tag{5}$$

where $\sigma = 0.3 \times d_{\text{target}}$.

Additionally, a hard constraint is enforced to eliminate infeasible routes. If the total route distance exceeds twice the target distance, the candidate is directly assigned a score of zero:

$$d(r) > 2 \cdot d_{\text{target}} \;\Rightarrow\; \text{Score}(r) = 0$$

This formulation ensures that routes close to the target distance are strongly preferred, while deviations are penalized smoothly rather than abruptly.

At the route level, candidate itineraries are evaluated using a weighted combination of multiple scoring components:

$$\begin{aligned}\text{Score}(r) = {} & w_1\, f_{\text{distance}}(r) \\ & + w_2\, f_{\text{variance}}(r) \\ & + w_3\, f_{\text{popularity}}(r) \\ & + w_4\, f_{\text{diversity}}(r) \\ & + w_5\, f_{\text{count}}(r)\end{aligned} \tag{6}$$

where r denotes a candidate route. Each component captures a complementary aspect of route quality. The distance term evaluates alignment with user-defined constraints, while the variance component measures the consistency of distances between consecutive POIs, favoring spatially coherent routes. The popularity term is computed as the average POI-level popularity within the route, normalized into the $[0, 1]$ range using the minimum and maximum popularity values observed across the entire dataset. This ensures that the score reflects the absolute attractiveness of the selected POIs rather than their relative ranking within a single route. The diversity component rewards routes that cover a broader range of POI subcategories, computed as the ratio of distinct subcategories present in the route to the maximum possible (four subcategory slots per POI). Finally, the count component promotes larger daily itineraries by considering the number of included POIs, scaled relative to the maximum allowed POIs per day.

To ensure a balanced contribution from all components, each term is defined within a comparable range. For instance, the distance component lies in the interval $(0, 1]$ due to its Gaussian formulation, while diversity and count are expressed as normalized ratios. This design prevents any single factor from dominating the overall score due to scale differences.

The weights are empirically determined to reflect the relative importance of each component:

$$(w_1, w_2, w_3, w_4, w_5) = (0.40, 0.20, 0.25, 0.05, 0.10)$$

corresponding to distance, segment variance, popularity, diversity, and POI count, respectively.

The diversity component is formally defined as:

$$f_{\text{diversity}}(r) = \min\left(\frac{|C(r)|}{4 \cdot |r|},\ 1\right) \tag{7}$$

where $C(r)$ denotes the set of distinct subcategory values present across the POIs of route r, and $|r|$ is the number of POIs in the route. The denominator reflects the maximum possible number of subcategory slots, since each POI carries up to four subcategory fields.

Overall, this composite scoring formulation enables the system to evaluate candidate routes holistically by jointly considering efficiency, spatial coherence, diversity, and attractiveness. As a

result, the selected itineraries achieve a balance between practical feasibility and user-oriented quality.
As described earlier, feasibility filtering is applied to the top-ranked candidates after scoring. If none of the top candidates satisfy both the duration and distance tolerance constraints, the highest-scoring candidate is retained as a fallback, preserving the prioritization induced by the scoring function.

4.4 Itinerary Generation Algorithm

The itinerary generation process follows an iterative, day-by-day construction strategy built on top of the candidate generation and scoring mechanisms described earlier. After the POI pool is formed based on user preferences, candidate routes are sampled, evaluated, and filtered according to feasibility constraints. Among these candidates, the highest-scoring route is selected as the itinerary for the current day.
Once a route is selected, its POIs are removed from the pool to prevent repetition across days. The same process is then repeated for subsequent days until the requested trip duration is satisfied. In this way, the system incrementally constructs a multi-day itinerary by selecting the best available route at each step.
At this stage, each daily plan consists of POIs. To obtain a concrete travel path, these POIs are passed to the routing engine (OSRM), which computes the shortest path between them based on the road network [34]. The resulting routes are then presented to the user through the interface, allowing each day's plan to be visualized as a navigable itinerary.
After the initial itinerary is generated and displayed, the system supports a flexible replanning workflow driven by user interaction. Users can modify the itinerary by adding, removing, or reordering POIs directly through the interface. In this phase, strict feasibility constraints are relaxed in favor of user control. The updated POI sequence is forwarded to the routing engine, and the recomputed route is presented back to the user.
This interaction loop allows users to iteratively refine their itineraries based on personal preferences and feedback, with each modification reflected in the interface. As a result, the system supports a dynamic and user-centered planning experience rather than a single static output.
The overall Monte Carlo-based candidate generation and selection procedure that underlies the itinerary construction process's pseudocode is provided in Appendix C.

4.5 Route Evaluation

The generated itineraries are evaluated through a combination of unit, integration, and system-level testing to ensure the correctness, consistency, and robustness of the underlying planning algorithms. These tests validate key properties such as constraint compliance, deterministic behavior under fixed inputs, and correct ranking among candidate routes.
In addition to automated testing, user-oriented evaluation was conducted to assess the practical quality of generated itineraries. Such evaluation complements algorithmic validation and will be presented in detail in the Testing and Evaluation section.

5 System Architecture

This section describes the implemented architecture of the GUIDE system from an implementation-oriented perspective. While the overall layered decomposition is defined in the High-Level and Low-Level Design documents, the purpose of this section is to explain how these architectural decisions are realized in the final system through concrete runtime components, backend modules, integration clients, and data access structures [26, 29]. The implemented system follows a layered and service-oriented organization composed of the Presentation Layer, Backend API Gateway Layer, Backend Services Layer, Integration Layer, and Data Layer. This decomposition preserves separation of concerns across user interaction, request handling, planning logic, external routing communication, and persistent data access, while also maintaining

traceability with the package structure and interface definitions presented in the Low-Level Design document.

5.1 Presentation Layer

The Presentation Layer is implemented as a React-based single-page web application responsible for collecting user inputs, invoking backend services through RESTful API calls, and rendering the generated itinerary and POI content to the user. This layer manages the full client-side interaction flow, including destination and category selection, trip duration and distance preference input, itinerary visualization, and interactive replanning operations such as POI addition, removal, and reordering.

In addition to user interaction, the Presentation Layer performs lightweight client-side validation and input normalization before requests are submitted to the backend. This includes restricting invalid selections, preventing incomplete submissions, and ensuring that request payloads remain compatible with the DTO structures expected by the API boundary. Once itinerary data is returned, the frontend renders route paths and POI markers using map-based visualization components and provides access to textual, visual, and audio content associated with each destination.

From the architectural perspective defined in the LLD, this layer corresponds to the frontend presentation and API client modules. Its implementation is intentionally kept separate from planning and routing logic; the frontend acts as a consumer of structured backend responses rather than embedding business rules locally. The frontend file and directory structure corresponding to this layer is provided in Appendix D.

5.2 Backend API Gateway Layer

The Backend API Gateway Layer is implemented as the controlled entry point of the backend and corresponds to the API Boundary package defined in the Low-Level Design. It is responsible for exposing the public REST interface of the system, mapping HTTP requests to controller operations, validating incoming payloads, and converting service outputs into consistent JSON responses. In the implemented system, this layer is realized through FastAPI-based controller components such as *RouteController* and *PoiController*, together with request validation and API-level error representation utilities.

Concretely, this layer handles endpoint-level operations for route generation, replanning, day suggestion, POI search, and POI content retrieval. Request bodies received through endpoints such as route generation and replanning are first mapped into DTO-like request structures, then checked for schema validity, required fields, value bounds, and feasibility-related constraints before being forwarded to core services. Similarly, warning and error cases are normalized into structured API responses rather than being exposed as raw backend exceptions.

Although this layer is referred to as an API Gateway layer in the architectural model, it is not implemented as a separate, standalone gateway service or reverse-proxy-based microservice. Instead, gateway responsibilities are incorporated into the backend API boundary itself. This design preserves architectural clarity while keeping the deployment model simple and ensuring that routing, validation, and response formatting remain centralized and consistent.

5.3 Backend Services Layer

The Backend Services Layer implements the main business logic of GUIDE and corresponds to the Core Services package defined in the Low-Level Design. After requests have been validated at the API boundary, this layer performs the end-to-end transformation from user preferences to a structured itinerary. Its responsibilities include POI pool retrieval, feasibility-aware itinerary construction, candidate generation, ranking and selection, route assembly preparation, and content aggregation for response formation.

The implemented service structure is decomposed into modular, stateless components such as *ItineraryService*, *PoiService*, *RoutingService*, and *ContentService*. Supporting components including *ItineraryBuilder*, *PlanRanker*, and

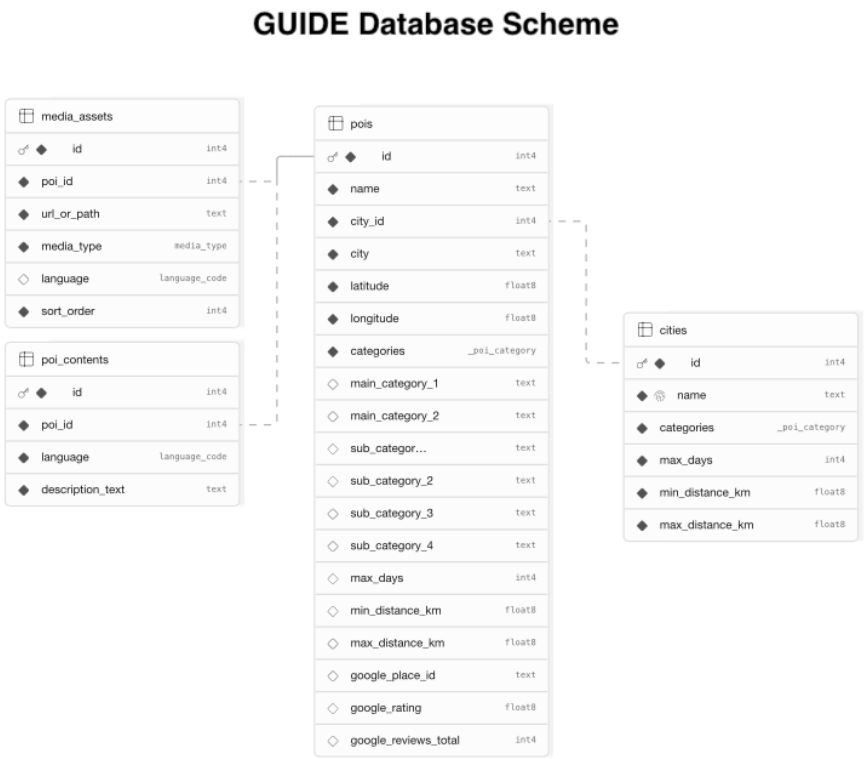

Figure 7: Database schema of the GUIDE system.

RouteAssembler further separate itinerary construction, scoring, and route composition concerns. This decomposition allows deterministic logic, heuristic ranking, and response assembly to evolve independently while still participating in a coherent request-processing pipeline.

A key implementation property of this layer is its statelessness at the service level. Backend services do not retain user-specific session state or previously generated itineraries between requests; instead, each request carries the full planning context required for processing. This design improves scalability, simplifies deployment, and makes the service layer more testable under both isolated and integration-level scenarios. The backend file and directory structure corresponding to the Backend API Gateway, Backend Services, and Integration layers is provided in Appendix E.

5.4 Integration Layer

The Integration Layer isolates communication with external or separately deployed subsystems from the core business logic. In the current implementation of GUIDE, this layer is centered on routing integration and encapsulates all interaction with the Open Source Routing Machine (OSRM) service used for real road-network-based distance, duration, and geometry computation.

This layer is implemented through dedicated client-side backend integration components, most notably the *OsrmClient*, which is responsible for constructing routing queries, issuing HTTP requests to the routing engine, and normalizing raw OSRM responses into internal route representations. By isolating these responsibilities within a dedicated integration module, the system prevents routing protocol details, response formats, and connection handling concerns from leaking into itinerary planning services.

The same abstraction is reused for both initial route generation and user-driven replanning flows. Once the selected or edited POI sequence is available, the integration layer transforms it into routing requests and returns normalized route data to the service layer for final response assembly. This separation improves modularity and also supports controlled testing, since routing communication

can be validated independently or replaced with mocks when necessary.

5.5 Data Layer

The Data Layer provides persistent storage and structured read access for the curated POI dataset, multilingual textual content, and static media assets used by the system at runtime. In the implemented architecture, this layer is not treated merely as a database schema; rather, it is realized through a combination of persistent relational storage, repository-oriented data access components, and shared domain data structures used across backend services.

The underlying relational schema, shown in Figure 7, is centered on the *pois* table, which stores core destination metadata such as geographic coordinates, category assignments, and popularity-related planning attributes. This core table is complemented by separate structures for multilingual textual content (*poi_contents*) and media asset references (*media_assets*), enabling content and asset retrieval without overloading core POI queries. City-level planning metadata is maintained separately through the *cities* table, allowing validation and planning constraints to be retrieved independently from POI-level content.

From the implementation perspective defined in the LLD, access to this layer is mediated through repository-style components such as *PoiRepository*, *ContentRepository*, *MediaRepository*, and *AudioAssetResolver*. These abstractions decouple service logic from storage-specific details and support graceful degradation in cases where some media assets are unavailable. In parallel, shared model structures provide a consistent representation of planning entities exchanged across the API boundary, core services, integration components, and response assembly pipeline.

This design allows the data layer to support both planning-time operations and user-facing content delivery while preserving modularity, maintainability, and future storage flexibility.

6 Implementation and Technologies

This section presents the implementation of the GUIDE system together with the technologies used across its components. The system is implemented as a combination of backend and frontend modules, whose implemented structural organization is provided in Appendix E and Appendix D. The implementation follows the architectural and design decisions defined in the previously developed documents, ensuring consistency between design and realization [28, 25, 26, 29]. In addition to implementation details, this section highlights the key technologies and external services integrated into the system.

6.1 Backend Implementation and Technologies

The backend of the GUIDE system is implemented in Python, which provides flexibility for rapid development and is well-suited for algorithmic workflows [37]. Core functionalities, including POI filtering, candidate route generation, scoring, and itinerary construction, are implemented as modular services aligned with the Backend Services Layer.

The implementation closely follows the algorithmic design presented in Section 4, where the itinerary generation process is realized through a pipeline of interacting service components. In this pipeline, POIs are first retrieved and filtered based on user inputs, followed by Monte Carlo-based candidate route generation, feasibility filtering using duration constraints, and final route selection through the composite scoring mechanism. These steps are implemented as separate but coordinated modules, enabling clear separation of concerns and maintainability of the system.

At the implementation level, this design is reflected in components detailed in Appendix E. This modular structure allows each stage of the planning process to be developed, tested, and refined independently while preserving a coherent overall workflow.

The system uses PostgreSQL as the underlying relational database system [36]. In the implemented

architecture, Supabase is adopted as a managed backend platform built on PostgreSQL, providing structured storage, API generation, and simplified data access [40]. This choice reduces the need for custom backend infrastructure while maintaining compatibility with standard relational database design.

In addition to structured data storage, multimedia assets such as images and audio files are hosted using Amazon Web Services (AWS) cloud storage solutions [1]. These assets are stored and served independently from the relational database, allowing efficient delivery of large media content without impacting query performance on core planning data. This separation between structured data and static asset storage improves scalability and supports optimized data access patterns within the system.

Version control is managed using Git, with the project hosted on GitHub [20, 21]. This setup supports collaborative development, version tracking, and reproducibility of the system.

Development and debugging were primarily carried out using the PyCharm integrated development environment, enabling efficient code organization and testing workflows [31].

6.2 Frontend Implementation and Technologies

The frontend is implemented using React, a widely used JavaScript library for building interactive user interfaces [8]. The application is developed using JavaScript, HTML, and CSS, and executed within a Node.js runtime environment during development and build processes [6].

The frontend is responsible for collecting user inputs, displaying generated itineraries, and visualizing routes on interactive maps. Communication with the backend is handled through RESTful API calls, with data exchanged in JSON format [17, 13].

Map-based visualization is implemented using the Leaflet.js library, which provides interactive map rendering, marker management, and route visualization capabilities [4]. Route geometries returned from the backend are rendered as polylines, and POIs are displayed as interactive markers, enabling users to explore generated itineraries in a spatial context.

Beyond basic rendering, the frontend implements an interactive planning and replanning workflow, where user actions such as adding, removing, or reordering POIs are reflected in the interface. This behavior is realized through a component-based state management approach, enabling dynamic updates without full page reloads.

This design directly supports the interactive nature of the GUIDE system, where the itinerary evolves through continuous user interaction rather than static output generation.

6.3 External Services Implementation and Technologies

The GUIDE system integrates several external services to support routing, content generation, and multimedia processing, allowing complex functionalities to be incorporated without reimplementing well-established infrastructure components.

For routing, the Open Source Routing Machine (OSRM) is used to compute travel distances and durations based on real road network data [34]. This enables accurate and scalable route computation while avoiding the need to develop a custom routing engine.

For content processing, the system leverages the Claude Haiku model developed by Anthropic for text refinement and multilingual transformation [12]. This integration represents a key design decision, as it incorporates a recent lightweight large language model into the system pipeline. The model is used to standardize, simplify, and translate POI descriptions, enabling consistent multilingual content generation without manual curation.

To ensure reliability and content quality, the outputs produced by the language model are not directly exposed at runtime. Instead, generated descriptions are reviewed and curated during dataset preparation, allowing the system to benefit from modern AI-based tooling while maintaining full

control over the final content presented to the user.
In addition, multilingual audio guides are generated using Google Text-to-Speech services, allowing scalable and automated production of audio content from textual descriptions [23]. Similar to text processing, audio content is pre-generated and validated, ensuring consistency and preventing unexpected runtime behavior.
Overall, these external services enable the system to integrate routing, content processing, and multimedia capabilities without reimplementing complex infrastructure components. By isolating these functionalities within the Integration Layer and offline data preparation pipeline, the system maintains modularity, reduces runtime complexity, and ensures consistent and reliable behavior during user interaction.

6.4 Testing Implementation and Technologies

The testing process of the GUIDE system was conducted across two distinct environments as defined in the Test Plan, ensuring both component-level validation and end-to-end system verification. Tests were executed in a controlled local development environment as well as an integrated system environment, allowing consistent validation across different execution contexts.
Backend testing was implemented using the *pytest* framework, which provides a flexible and scalable structure for unit and integration testing in Python. API-level testing was performed using FastAPI's *TestClient*, enabling simulation of HTTP requests and validation of endpoint behavior without requiring a running server instance. In addition, the *httpx* library was used for asynchronous and real-world request simulation, particularly for integration-level testing scenarios.
Performance-related tests were supported by built-in Python modules such as *time* and *statistics*, allowing measurement and analysis of response times across multiple executions. System resource behavior was monitored using modules such as *gc* for memory management observations and *os* for environment-level interactions. Concurrent execution scenarios were evaluated using the *threading* module to simulate multiple request flows.
Input validation and schema correctness were verified using *pydantic*, ensuring that invalid or malformed requests were properly detected and handled at the API boundary. This aligns with the system's validation and error-handling strategy.
Frontend-related testing focused on functional validation of user interaction flows and cross-device compatibility, as defined in the user and system testing scenarios. These tests ensured that frontend-backend integration, state updates, and interactive behaviors operated correctly under different usage conditions.
Overall, the testing process combines automated backend validation, API-level simulation, performance measurement, and scenario-based system testing, providing comprehensive coverage across different layers of the system.

6.5 Use of Modern and Emerging Technologies

The GUIDE system incorporates several modern technologies to enhance functionality while maintaining a controlled and reliable runtime behavior. These technologies have been introduced and discussed in their respective contexts in earlier sections.
Supabase is adopted as a backend-as-a-service platform built on PostgreSQL, reducing infrastructure overhead while preserving compatibility with relational data models [40, 36]. In addition, the Claude Haiku model developed by Anthropic is used during offline data preparation for text normalization and multilingual transformation [12]. As discussed in the Dataset section, model outputs are curated before integration, ensuring consistency and preventing unpredictable runtime behavior.
Similarly, Google Text-to-Speech services are used to generate multilingual audio content in advance [23]. As previously described, audio assets are precomputed and stored, enabling efficient and low-latency delivery during runtime without introducing external dependencies into

the request-processing pipeline.
Overall, the system follows a hybrid design approach, where modern tools are leveraged for capability enhancement while runtime execution remains deterministic and decoupled from external service dependencies.

7 Testing and Evaluation

This section presents the testing and evaluation of the GUIDE system based on the Test Plan Report [30]. The objective of this phase is to verify that the implemented system satisfies the defined functional, architectural, and performance requirements.

7.1 Testing Methodology

The testing methodology of the GUIDE system follows a multi-level verification strategy defined in the Test Plan Report [30]. Testing is structured into five complementary levels: unit testing, integration testing, system testing, performance testing, and user testing. Each level targets a different abstraction of the system and collectively ensures correctness, robustness, and usability.

7.1.1 Unit Testing

Unit testing focuses on validating individual components in isolation. Core modules such as validation logic, POI filtering, Monte Carlo itinerary generation, scoring functions, replanning logic, routing client interactions, and content aggregation are tested using deterministic inputs, with external dependencies replaced by mocks to ensure reproducibility and strict isolation.
In total, 29 unit test cases were designed and executed, covering the majority of unit-level functionalities. The test set includes both typical scenarios and edge cases, ensuring reliable and consistent behavior of individual components across different input conditions.
A brief summary of the unit test cases is provided in Table 1. The full definitions of these test cases, including detailed inputs, expected outputs, and execution-level descriptions, are available in the Test Plan Report [30].

Test ID	Brief Description
TC-UT-01	Validates that a request is rejected when the city field is missing.
TC-UT-02	Validates that a request is rejected when categories are missing or empty.
TC-UT-03	Ensures that at least one planning constraint (days or distance) is required.
TC-UT-04	Verifies that tripDays does not exceed the defined maximum limit.
TC-UT-05	Ensures that daily distance does not exceed city-specific limits.
TC-UT-06	Confirms that POI filtering returns only POIs from the selected city.
TC-UT-07	Confirms that POI filtering returns only POIs within selected categories.
TC-UT-08	Ensures that generated candidate itineraries contain no duplicate POIs.
TC-UT-09	Verifies that the scoring function produces deterministic results.
TC-UT-10	Ensures that the highest-scoring candidate is always selected.
TC-UT-11	Verifies that the system returns a warning when POIs are insufficient.
TC-UT-12	Ensures that no POI is assigned to more than one day.
TC-UT-13	Validates correct parsing of OSRM response fields.
TC-UT-14	Ensures final route output satisfies all request constraints.
TC-UT-15	Verifies correct mapping between route segments and itinerary days.
TC-UT-16	Ensures route totals match the sum of segment values.
TC-UT-17	Validates correct application of remove operations in replanning.
TC-UT-18	Validates correct application of reorder operations in replanning.
TC-UT-19	Validates correct application of add operations in replanning.
TC-UT-20	Ensures replanning does not affect unrelated days.
TC-UT-21	Ensures routing is recomputed only for affected segments.
TC-UT-22	Verifies that replanning preserves constraint compliance.
TC-UT-23	Ensures invalid replanning edits are rejected correctly.
TC-UT-24	Validates that textual content is returned for POIs.
TC-UT-25	Validates that image content is returned when available.
TC-UT-26	Validates that audio content is returned for supported languages.
TC-UT-27	Ensures missing images are handled gracefully.
TC-UT-28	Ensures missing audio is handled gracefully.
TC-UT-29	Ensures batch content retrieval preserves correct mapping.

Table 1: Summary of unit test cases.

7.1.2 Integration Testing

Integration testing evaluates interactions between architectural layers, including Presentation–API, API–Core Services, Core Services–Data Access, and Core Services–Routing integration. These tests ensure that data flows correctly across system boundaries and that contracts between components are preserved.

In total, 17 integration test cases were designed and executed. While the system allows a large number of possible integration combinations, the selected test cases focus on representative interaction paths that capture the critical data flow within the system pipeline.

Test ID	Brief Description
TC-IT-01	Validates correct mapping from frontend request to backend DTO.
TC-IT-02	Ensures correct unit conversion across frontend and backend boundaries.
TC-IT-03	Validates stability of error response structure.
TC-IT-04	Validates stability of warning response structure.
TC-IT-05	Ensures correct invocation flow between API and core services.
TC-IT-06	Ensures validation failures prevent core execution.
TC-IT-07	Validates propagation of warnings through the system layers.
TC-IT-08	Ensures data layer returns valid POI entities.
TC-IT-09	Validates detection of category mapping mismatches.
TC-IT-10	Ensures missing content does not break planning pipeline.
TC-IT-11	Validates correct routing response for map rendering.
TC-IT-12	Ensures routing failures are handled gracefully.
TC-IT-13	Validates consistency of replanning pipeline outputs.
TC-IT-14	Ensures recomputation occurs only for affected segments.
TC-IT-15	Validates stability of content response with missing media.
TC-IT-16	Validates language-based audio selection and fallback behavior.
TC-IT-17	Ensures batch content retrieval preserves correct mapping.

Table 2: Summary of integration test cases.

Specifically, the chosen tests target key integration points such as request handling from the frontend to the backend, transformation of API inputs into internal service calls, interaction between planning modules and the data layer, and integration with the external routing service. These points were selected because they represent the most failure-prone boundaries where inconsistencies, data mismatches, or contract violations are most likely to occur.

These integration tests serve as an intermediate validation step prior to system-level evaluation. While they focus on specific interaction boundaries, they are complemented by system testing, where complete end-to-end workflows inherently exercise all integrated components of the system. Together, integration and system testing ensure full coverage of component interactions, providing both early detection of interface-level issues and comprehensive validation of overall system behavior.

A brief summary of the integration test cases is provided in Table 2. The full definitions and details of these test cases are available in the Test Plan Report [30].

7.1.3 System Testing

System testing validates complete end-to-end workflows under realistic usage scenarios. These tests simulate full user interactions, including itinerary generation, route visualization, content exploration, and replanning.

In total, 7 system test cases were designed and executed. Each test represents a full workflow scenario, covering the entire system pipeline from user input to final output.

These tests validate that the system operates correctly as a unified whole, covering the complete flow from user input to final output. By executing full workflows, system testing confirms the correct coordination of all components under realistic usage conditions.

A brief summary of the system test cases is provided in Table 3. The full definitions and details of these test cases are available in the Test Plan Report [30].

Test ID	Brief Description
TC-ST-01	Validates full end-to-end itinerary generation workflow.
TC-ST-02	Validates user-facing insufficiency handling in end-to-end flow.
TC-ST-03	Validates distance preference propagation in final routes.
TC-ST-04	Validates end-to-end replanning updates and visualization.
TC-ST-05	Validates adding POIs via replanning without affecting other days.
TC-ST-06	Validates complete workflow including planning, exploration, and replanning.
TC-ST-07	Validates accessibility and correctness of transparency page.

Table 3: Summary of system test cases.

7.1.4 Performance Testing

Performance testing assesses system responsiveness and resource usage under both typical and upper-bound scenarios. In total, 7 performance test cases were executed to evaluate system behavior under different load conditions.
These tests focus on verifying that itinerary generation and related operations complete within acceptable latency limits, ensuring that the system remains responsive in real-world usage. Since the system is designed for interactive use, maintaining stable and predictable response times is essential for a smooth user experience.
A brief summary of the performance test cases is provided in Table 4. The full definitions and details of these test cases are available in the Test Plan Report [30].

Test ID	Brief Description
TC-PT-01	Validates that typical usage latency remains within defined limits.
TC-PT-02	Ensures system stability under upper-bound configurations.
TC-PT-03	Ensures content retrieval does not dominate response time.
TC-PT-04	Ensures replanning latency is comparable to full planning.
TC-PT-05	Validates stable memory usage under repeated execution.
TC-PT-06	Ensures CPU usage remains proportional to workload.
TC-PT-07	Ensures audio retrieval does not block main interaction flow.

Table 4: Summary of performance test cases.

7.1.5 User Testing

User testing complements technical validation by evaluating usability and perceived itinerary quality through structured user feedback. A total of 12 user testing cases were conducted, focusing on user satisfaction, route coherence, and overall interaction quality. The evaluation was carried out with a participant group of **40 users**.
The primary objective of user testing is to assess whether the generated itineraries are not only technically valid but also meaningful and appealing from a user perspective. While the system ensures structural correctness through algorithmic design, user testing provides insight into whether the resulting routes align with user expectations and preferences.
This evaluation is particularly important because itinerary quality involves subjective judgment, and certain aspects of route usefulness cannot be fully captured through deterministic testing alone.
A brief summary of the user test cases is provided in Table 5. The full definitions and details of these test cases are available in the Test Plan Report [30].

Test ID	Brief Description
TC-UST-01	Validates mean score threshold for Scenario 1.
TC-UST-02	Validates agreement ratio threshold for Scenario 1.
TC-UST-03	Validates mean score threshold for Scenario 2.
TC-UST-04	Validates agreement ratio threshold for Scenario 2.
TC-UST-05	Validates mean score threshold for Scenario 3.
TC-UST-06	Validates agreement ratio threshold for Scenario 3.
TC-UST-07	Validates overall usability mean score threshold.
TC-UST-08	Validates overall usability agreement ratio threshold.
TC-UST-09	Validates real-world usage intention mean score threshold.
TC-UST-10	Validates real-world usage intention agreement ratio threshold.
TC-UST-11	Ensures no critical recurring usability issue is present.
TC-UST-12	Ensures no extreme variability in usability perception.

Table 5: Summary of user test cases.

7.2 Test Results

7.2.1 Quantitative Test Results

The execution of the test suite resulted in the successful completion of all defined test cases. As summarized in Table 6, a total of 72 tests were conducted across unit, integration, system, performance, and user testing categories, and all tests passed without any remaining failures.

Test Category	Total	Passed	Failed
Unit Testing	29	29	0
Integration Testing	17	17	0
System Testing	7	7	0
Performance Testing	7	7	0
User Testing	12	12	0
Total	**72**	**72**	**0**

Table 6: Summary of test results.

During the development process, testing played an active role in identifying and resolving issues rather than being used solely for final validation. In earlier iterations, several inconsistencies and edge-case problems were observed, including constraint validation mismatches, boundary handling issues in itinerary generation, and minor data flow inconsistencies between components. These issues were systematically addressed through iterative refinements such as improving validation rules, correcting scoring behavior, and stabilizing interactions between modules. A more detailed discussion of the identified issues and the corresponding resolutions is provided in Section 8.

The final results demonstrate that all identified issues were successfully resolved and that the system reaches a stable and consistent state. While Table 6 provides an overall summary of test outcomes, the detailed status of each individual test case, including pass/fail results, is explicitly documented in Table 1 of the Test Plan Report [30]. These results collectively indicate that the system behaves reliably across both individual components and complete end-to-end workflows.

In addition to the test outcomes, code coverage analysis was conducted to evaluate the extent to which the implemented test suite exercises the system components. The overall test coverage was measured as **85%**, indicating that the majority of critical modules are effectively tested.

Coverage was particularly high in core components such as validation logic, itinerary planning, and service-level operations, while comparatively lower coverage was observed in less critical or integration-heavy components.

7.2.2 User Evaluation Results

A user-centered evaluation was conducted with 40 participants to assess the usability, clarity, and perceived quality of the GUIDE system. The evaluation was carried out using a structured Likert-scale questionnaire distributed via Google Forms, covering key aspects of itinerary generation, visualization, replanning behavior, and overall system interaction.

The evaluation was designed with predefined acceptance criteria. Specifically, the average score for each question was expected to exceed 3.5, and at least 70% of responses were expected to fall within the “Agree” (4) or “Strongly Agree” (5) categories. The collected responses were analyzed both quantitatively and qualitatively to assess system performance from a user perspective.

Evaluation Criterion	4 (%)	5 (%)	4+5 (%)	Avg.
Itinerary clarity	17.5	82.5	100.0	4.83
Itinerary structure	25.0	70.0	95.0	4.63
Map visualization clarity	7.5	90.0	97.5	4.88
POI content usefulness	7.5	87.5	95.0	4.83
Replanning correctness	10.0	87.5	97.5	4.85
Post-change consistency	10.0	87.5	97.5	4.85
Overall usability	15.0	82.5	97.5	4.80
Expectation satisfaction	17.5	75.0	92.5	4.68
Real-use intention	10.0	82.5	92.5	4.73
System stability	17.5	77.5	95.0	4.74
Interaction flexibility	15.0	75.0	90.0	4.67
Overall	**13.86**	**81.59**	**95.45**	**4.77**

Table 7: User evaluation results based on Likert-scale responses.

The results presented in Table 7 clearly demonstrate that all evaluation criteria significantly ex-

ceed the predefined acceptance thresholds. While the evaluation was designed with a minimum requirement of an average score above 3.5 and at least 70% of responses in the "Agree" (4) and "Strongly Agree" (5) categories, all questions achieved substantially higher values.
Specifically, the lowest average score observed across all questions is 4.63, and the minimum proportion of "Agree" and "Strongly Agree" responses is 90.0%, both of which are well above the target thresholds. The overall average score of 4.77 and 95.45% positive responses further indicate a consistently high level of user satisfaction.
These results confirm that the system not only meets but significantly surpasses its usability and user acceptance goals, demonstrating strong performance across all evaluated dimensions.

Qualitative Feedback Analysis In addition to the quantitative results, participants provided open-ended feedback highlighting both strengths of the system and potential areas for improvement.
Positive feedback consistently emphasized the clarity of itinerary generation, the usefulness of POI descriptions, and the overall practicality of the system for real-world trip planning. Several participants described the system as intuitive, well-structured, and engaging, indicating that the core user experience aligns well with user expectations.
Constructive feedback primarily focused on usability improvements and feature extensions. A number of participants suggested simplifying certain user interface interactions, particularly in category selection and navigation between planning steps. Additionally, requests were made for extended functionality, including support for additional countries, traffic-aware routing, enhanced map interaction, and improved visibility of route-related details.
Some technical observations were also reported, such as minor inconsistencies in duration display, redundant API calls in specific views, and improvements related to system feedback mechanisms (e.g., clearer error or status messages). These issues were identified during the evaluation phase and subsequently resolved through targeted refinements and optimizations. As a result, they do not affect the final version of the system.
Overall, the qualitative feedback is strongly aligned with the quantitative results, confirming that the system provides a usable and valuable experience while also identifying clear directions for future improvement.
These findings collectively demonstrate that the system not only achieves its functional requirements but also delivers a user experience that is perceived as reliable, intuitive, and practically useful. The alignment between quantitative metrics and qualitative feedback further strengthens the validity of the evaluation results.

7.3 Evaluation of Results

While the test results demonstrate that the GUIDE system satisfies its functional, integration, performance, and user-centered requirements, the evaluation must be interpreted within the inherent limitations of the testing strategy and the nature of the problem domain.

7.3.1 Heuristic Evaluation Limitations

The itinerary generation process is based on heuristic methods, which makes it impractical to formally verify the global optimality of generated routes. The testing framework ensures correctness, constraint compliance, and consistent ranking within candidate sets; however, it does not guarantee that the selected itinerary is the best possible solution in the full combinatorial space. This reflects a deliberate design choice, where the system prioritizes feasible and high-quality solutions over provable global optimality.
To partially address this limitation, user testing was incorporated as a complementary evaluation mechanism. However, such evaluation remains inherently subjective and is interpreted by the project team as not constituting a formal measure of optimality.

7.3.2 User Testing Constraints

User testing was conducted to evaluate qualitative aspects of the system, including the perceived attractiveness of generated itineraries, overall usability, and user satisfaction with the planning experience. In particular, it aimed to capture whether users found the generated routes meaningful, coherent, and aligned with their expectations.

User testing was conducted on a relatively small sample, primarily consisting of individuals within the project team's extended social circle. In addition to the limited sample size, the participant group was relatively homogeneous in terms of background, with many participants sharing similar educational or technical profiles. While this provided useful qualitative feedback, the sample is not fully representative of a broader and more diverse user population.

As a result, user testing outcomes may contain implicit bias due to participant familiarity with the project team and the lack of diversity in the sample. Consequently, the findings cannot be directly generalized to all potential users. A larger-scale and more heterogeneous evaluation would be required for a more reliable assessment of user satisfaction and itinerary quality.

7.3.3 Dataset and Media Evaluation Gaps

Although the dataset was manually reviewed for structural correctness and relevance, certain qualitative aspects were not exhaustively tested. In particular, while images were checked for correctness and relevance, this process relied on manual inspection performed by a limited number of reviewers. As a result, the evaluation of visual content was not based on a systematic or standardized methodology, and attributes such as visual quality and representativeness were not assessed in a consistent and reproducible manner.

Similarly, audio content generated via text-to-speech systems was not subjected to comprehensive verification. While samples were informally reviewed, no structured evaluation procedure was applied to validate pronunciation accuracy or linguistic quality across all entries. Issues such as pronunciation inaccuracies (especially for proper nouns across different languages) may therefore still occur.

These limitations highlight that multimedia validation was primarily manual and exploratory rather than fully automated or rigorously standardized. A more reliable testing approach would require scalable evaluation mechanisms, such as multi-reviewer validation, standardized quality metrics, or automated quality assessment pipelines.

7.3.4 External Dependency Testing Limitations

Routing functionality relies on an external routing engine, and testing focuses on correct integration, response handling, and data consistency. However, the geographic accuracy or real-world optimality of the computed routes is not independently verified within the system.

Therefore, routing quality is treated as a trusted external dependency, and its evaluation remains outside the direct scope of the testing process. From the perspective of the project team, additional validation strategies could be incorporated to strengthen this aspect, such as cross-verifying routes with alternative routing engines or comparing results against independent mapping services. These approaches could provide a more robust assessment of routing accuracy beyond single-source dependency.

Overall, the evaluation demonstrates that the system is technically reliable and functionally correct within its defined scope, while also making clear that certain dimensions (particularly those involving subjective perception, dataset quality, and external dependencies) cannot be fully validated through deterministic testing alone. From the perspective of the project team, these limitations are not defects but inherent characteristics of the problem domain.

For real-world deployment, addressing these aspects would require larger-scale user studies, more diverse datasets, and additional validation mechanisms, particularly for subjective quality and external service outputs.

8 Observed Issues and Resolutions

During the development and testing phases, several issues and inconsistencies were identified across different components of the system. These issues primarily emerged in edge cases, boundary conditions, and component interactions, and were systematically addressed through iterative refinement. This section summarizes the key issues encountered and the corresponding resolutions.

8.1 Constraint Handling and Edge Case Issues

Issue: Certain edge cases related to constraint handling caused inconsistencies between expected and actual system behavior. For example, when users provided a minimum distance value of zero, the system produced runtime errors instead of generating a valid itinerary.
Cause: Although the design documents specified that such inputs should be normalized (e.g., converting zero values to a minimum threshold), this behavior was not consistently implemented in the backend logic. Additionally, inconsistencies were observed between documented behavior (warning vs. error handling) and actual system responses.
Resolution: Input validation and normalization logic were revised to ensure that all constraint values are mapped to valid operational ranges. Error and warning mechanisms were also aligned with the documented specifications.
Outcome: The system now handles boundary conditions robustly, ensuring consistent behavior across all constraint inputs.
Responsible: Ebrar Sude Doğan

8.2 Algorithmic Tuning and Scoring Imbalance

Issue: The initial scoring function used in the Monte Carlo-based route generation led to unintuitive route selections, where highly popular POIs were favored even if they significantly violated distance constraints.
Cause: The weight assigned to the distance component in the scoring function was relatively low compared to popularity, resulting in routes that were excessively long (e.g., up to 3–4 times the desired distance).
Resolution: The scoring function was iteratively refined by adjusting weight parameters and rebalancing the contribution of distance, popularity, and diversity components. Multiple experimental runs were conducted to observe the impact of different configurations.
Outcome: The updated scoring mechanism produces more balanced itineraries that better respect user-defined constraints while maintaining route quality.
Responsible: Tuna Kodal

8.3 Routing Integration Issues (OSRM)

Issue: The routing engine did not always produce the expected shortest path ordering when given an unordered set of POIs. Instead of returning an optimized sequence, the system produced suboptimal route structures.
Cause: It was initially assumed that the routing service would internally optimize the order of input POIs. However, the API required an ordered sequence and did not perform global route optimization over unordered inputs.
Resolution: A preprocessing step was introduced to order POIs before sending them to the routing engine. This was achieved using a heuristic based on approximate (Haversine) distances to generate a reasonable initial ordering.
Outcome: The routing process now produces consistent and geographically coherent routes, aligning with expected behavior.
Responsible: Tuna Kodal, Kayrahan Toprak Tosun

8.4 Data Consistency and Mapping Issues

Issue: Inconsistencies between backend data representations and frontend expectations led to failures in data retrieval and filtering, particularly for category and city names.
Cause: Differences in naming conventions between database fields and frontend components resulted in mismatches during API communication.
Resolution: A mapping (adapter/assembler)

layer was introduced to standardize and translate identifiers between backend and frontend representations.
Outcome: Data flow between system components became consistent and reliable, eliminating integration-related mismatches.
Responsible: Kayrahan Toprak Tosun

8.5 Dataset Quality and External Data Issues

Issue: Raw data collected from external sources contained significant noise and inconsistencies, including irrelevant POIs, low-quality descriptions, and unsuitable multimedia content (e.g., personal photos).
Cause: Sources such as OpenStreetMap and Wikipedia include user-generated content, which varies widely in quality and reliability.
Resolution: A multi-stage data cleaning and validation pipeline was implemented, combining automated filtering with extensive manual curation. Irrelevant entries were removed, descriptions were refined, and multimedia content was manually validated.
Outcome: The dataset was transformed into a high-quality, tourism-oriented collection suitable for itinerary generation and user-facing presentation.
Responsible: Erdem Baran, Ebrar Sude Doğan

8.6 User Interface and Responsiveness Issues

Issue: During user testing, layout and responsiveness issues were observed, particularly in mobile views, where certain components were not displayed correctly.
Cause: Media query definitions and responsive design rules were not fully optimized during the initial frontend implementation.
Resolution: Frontend styles were revised, and responsive design rules were improved to ensure consistent behavior across different screen sizes.
Outcome: The user interface now provides a stable and consistent experience across both desktop and mobile environments.
Responsible: Erdem Baran
Overall, these issues were identified and resolved prior to the final testing phase, and no critical defects remained in the final system. The iterative resolution of these problems contributed significantly to the robustness and reliability of the GUIDE system.

9 Impact of the Engineering Solution

This section evaluates the broader impact of the GUIDE system from multiple engineering perspectives, including economic, societal, environmental, legal and ethical, and security-related dimensions. The discussion is grounded not only in the intended purpose of the system, but also in concrete architectural decisions, dataset design choices, and observations obtained during testing and evaluation. Where direct measurement is not available, the impact is interpreted as a technically justified consequence of the implemented design rather than as a formally quantified outcome.

9.1 Economic Impact

From an engineering perspective, GUIDE reduces travel-planning overhead by consolidating multiple tasks (POI discovery, itinerary generation, route construction, and content access) within a single system. This integration decreases the need for users to switch between separate platforms and therefore reduces both interaction cost and planning effort.
The economic impact of this design can also be considered at the level of regional tourism visibility. Since the itinerary generation process is not limited to a fixed set of highly popular landmarks, the system can expose users to additional culturally or geographically relevant POIs that may not have been part of their initial plan. User evaluation results indicate that the generated itineraries were generally perceived as useful and aligned with expectations, suggesting that the system can encourage broader exploration behavior beyond the most obvious destinations.
In addition, the system was intentionally designed around open-source tools, curated offline datasets, and low-cost service dependencies. Architectural choices such as preprocessing content, pre-

generating multimedia assets, and limiting expensive routing computations to selected routes reduce operational overhead. This design improves cost-efficiency and makes the solution more sustainable within the scope of an academic prototype and a low-budget deployment model.

9.2 Societal Impact

GUIDE contributes to broader access to cultural and historical information by combining structured itinerary generation with multilingual content delivery in Turkish, English, and German. This enables the system to serve both domestic and international users within a common interaction model.

The inclusion of textual, visual, and audio-based destination content also improves accessibility from a user-experience perspective. In particular, pre-generated audio guides support users who may prefer listening over reading or who benefit from multimodal interaction. User evaluation results and open-ended feedback indicate that participants found the system understandable, usable, and informative, supporting the view that the platform improves access to cultural information in a practically usable form.

At a broader level, GUIDE also encourages more structured and informed travel behavior. Instead of requiring users to manually aggregate scattered online information, the system presents curated and route-aware outputs within a single workflow. This reduces cognitive fragmentation and supports a more guided interaction with cultural and historical destinations.

9.3 Environmental Impact

The environmental relevance of GUIDE arises primarily from its route construction logic. The system explicitly incorporates distance-related constraints and route coherence into candidate evaluation, which encourages more compact and spatially efficient itineraries. Performance and system-level test results confirm that generated itineraries remain aligned with target distance ranges and avoid unnecessarily dispersed route structures.

Although direct environmental metrics such as fuel consumption or carbon emissions were not measured, the underlying engineering implication is clear: routes that avoid excessive travel distance are likely to reduce unnecessary movement between scattered destinations. In this sense, the system supports a more resource-efficient travel pattern by favoring localized and distance-aware exploration.

This impact should therefore be interpreted as an indirect but technically grounded consequence of the implemented routing and scoring strategy, rather than as a directly quantified environmental measurement.

9.4 Legal and Ethical Impact

The legal and ethical impact of GUIDE is closely tied to its data handling model and content sourcing strategy. The system is designed to minimize privacy risk by operating without persistent user accounts and without retaining user-specific planning data beyond the immediate interaction context. This session-based design reduces exposure to personal data processing concerns and aligns with data minimization principles reflected in GDPR and KVKK-oriented design reasoning [38, 16].

This design choice is reinforced by the technical structure of the system: planning and replanning requests are processed in a stateless manner at the service level, and no persistent user profile is required for core functionality. As a result, the system achieves its primary functionality without depending on long-term user tracking or behavioral profiling.

In addition, the project relies on third-party data and content sources such as OpenStreetMap, Wikipedia, Google Places, and Google Text-to-Speech [7, 11, 2, 23]. Their use is bounded by attribution, licensing, and controlled integration practices. Content is not accepted blindly at runtime; instead, external data is incorporated through preprocessing, validation, and curation stages. This reduces the risk of propagating misleading, inappropriate, or low-quality content directly to the user.

Ethically, the recommendation logic is also designed to remain transparent and non-commercial. Route generation is driven by explicit scoring criteria and user-defined constraints rather than by sponsorship, paid prioritization, or platform bias. In that sense, the system supports neutrality and user-centeredness as explicit engineering properties rather than incidental outcomes.

9.5 Security Impact

From a security perspective, GUIDE adopts a minimal-exposure architecture. The system avoids persistent storage of sensitive user information, limits its runtime dependency surface, and relies on controlled communication with external services. This reduces both the amount of sensitive data handled by the application and the attack surface associated with user-specific state.

All client–server and service integration interactions are designed to operate over secure communication channels, and the architecture avoids unnecessary retention of personally identifying information. In addition, separating structured POI data from static media storage contributes to cleaner access boundaries and reduces coupling between core planning logic and auxiliary content delivery.

Integration testing also verified that external communication paths, including routing interactions and response handling, operate in a controlled manner under both normal and failure scenarios. Although the system does not implement a comprehensive enterprise-grade security model, the adopted practices provide an appropriate baseline level of security for the project scope while preserving architectural simplicity and reliability.

10 Contemporary Issues

This section discusses contemporary trends and challenges in digital travel planning, and positions the GUIDE system within the context of increasing consumption, information overload, automation, and data privacy concerns.

10.1 Increasing Consumption and Experience-Oriented Travel

In recent years, there has been a noticeable increase in consumption-oriented travel behavior, where individuals aim to visit more places, experience more activities, and maximize the number of destinations within a limited time. This trend is further amplified by the widespread use of social media, where sharing travel experiences has become a central motivation for many users.

GUIDE is developed within this context, addressing the growing demand for structured and efficient travel planning. By organizing destinations into coherent itineraries, the system responds to the need for managing increasingly dense and experience-driven travel expectations.

10.2 Platform Neutrality and Bias-Free Recommendations

With the increasing commercialization of digital platforms and the rise of cross-platform partnerships, many travel-related services promote specific locations through sponsored content, partnerships, or algorithmic prioritization. Restaurants, venues, or attractions may gain visibility not based on relevance, but through commercial agreements.

GUIDE adopts a fundamentally different approach by remaining independent from such collaborations. The system does not prioritize or promote any Point of Interest based on external partnerships or commercial incentives. Instead, all recommendations are generated solely based on defined constraints and transparent scoring mechanisms. This ensures that the produced itineraries remain unbiased, consistent, and aligned with user-defined preferences rather than external influence.

10.3 Acceleration and Demand for Instant Solutions

Modern users increasingly expect fast and automated solutions, influenced by the rise of digital services and AI-based tools. There is a clear shift toward obtaining maximum output with minimal

input, reflecting a broader trend toward faster and more effortless digital interactions.
GUIDE aligns with this expectation by providing rapid itinerary generation with minimal user effort. At the same time, the system maintains a controlled structure rather than relying entirely on fully automated or opaque decision-making processes.

10.4 Information Overload and Limited Attention Span

One of the most prominent challenges in contemporary digital environments is the overwhelming amount of available information. Travel-related queries often return excessive and unstructured results, making it difficult for users to identify relevant and reliable content.
At the same time, users' attention spans have significantly decreased, reducing their willingness to engage with long-form content. GUIDE addresses this issue by providing curated, concise, and structured information. Content is intentionally designed to be brief and easily consumable, allowing users to quickly access relevant insights without being overwhelmed.

10.5 Data Quality and Content Reliability

The abundance of user-generated and unverified content on the internet introduces significant variability in data quality. Inconsistent or misleading information can negatively affect travel planning decisions.
To mitigate this issue, GUIDE emphasizes curated and standardized data. POI information is carefully processed and structured to ensure consistency, providing users with a more reliable foundation for decision-making compared to unfiltered online sources.

10.6 AI and Automation in Decision-Making

Modern systems increasingly rely on AI-driven approaches for decision-making and recommendation generation. While these approaches offer flexibility and scalability, they may also introduce challenges related to transparency, control, and predictability.
GUIDE adopts a more controlled approach by combining algorithmic and heuristic methods rather than fully relying on opaque AI models. This reflects a conscious design decision to balance automation with interpretability and system reliability.

10.7 Privacy and Data Minimization

Privacy has become a critical concern in modern digital systems, as many platforms extensively collect and process user data through tracking mechanisms such as cookies and behavioral analytics.
GUIDE takes a contrasting approach by minimizing data collection. The system operates without persistent user accounts and does not store user-specific information beyond the immediate interaction. This reduces privacy risks and aligns with growing awareness around data protection, although it also limits certain personalization capabilities.

10.8 User-Centered Design Considerations

The GUIDE system is designed with a user-centered perspective, aiming to reduce cognitive load and simplify decision-making in travel planning.
Instead of requiring users to manually combine multiple tools, GUIDE provides a structured and integrated workflow. This reduces planning complexity and enables users to focus on exploration rather than organization.
Additionally, the system balances automation and user control. While itineraries are automatically generated, users retain the ability to modify and personalize their plans through replanning features. This hybrid approach ensures both efficiency and flexibility.

11 Discussion

The GUIDE system demonstrates that a unified, constraint-aware travel planning solution can be successfully designed and implemented within the scope of an undergraduate engineering project.

Despite being developed within a limited timeframe, by a four-member team, and under academic constraints, the system integrates multiple complex components into a coherent and functional platform.
One of the main strengths of the project is its end-to-end design. Rather than addressing isolated parts of the travel planning process, GUIDE brings together data collection, planning, visualization, and content delivery within a single workflow. In addition, the system achieves interactive performance through careful design decisions, such as heuristic route generation and controlled use of external services. These choices allow the system to remain both responsive and practically usable.
At the same time, it is important to recognize that the system operates under several constraints that directly influenced its design and implementation. These constraints are not simply missing features, but rather reflect conscious trade-offs made due to limitations in time, resources, and problem complexity. A detailed discussion of these constraints is provided in the Project Specification, Low-Level Design, and Test Plan documents [28, 29, 30].

11.1 Constraints

11.1.1 Algorithmic and Performance Constraints

The itinerary generation problem is inherently combinatorial, and computing globally optimal solutions is computationally expensive. In order to maintain an interactive response time (approximately 5 seconds), the system adopts a heuristic, sampling-based approach.
As a result, while the system produces high-quality and feasible routes, it does not guarantee global optimality. This limitation is not due to incomplete implementation, but rather a deliberate trade-off between solution quality and computational efficiency.

11.1.2 Geographic Scope Constraints

The system is limited to Türkiye, primarily due to the significant effort required to construct a high-quality POI dataset. Extending the system to additional regions would require repeating the entire data collection, cleaning, categorization, and validation pipeline.
Given the project timeline and team capacity, focusing on a single country enabled higher data quality and consistency, at the cost of reduced geographic coverage.

11.1.3 Data and Real-Time Information Constraints

GUIDE operates on a curated but largely static dataset. Real-time data such as traffic conditions, POI operational status (e.g., opening hours, temporary closures), and dynamic environmental factors are not incorporated.
The primary reasons for this limitation are:
(i) the fact that high-quality real-time data sources are typically provided through paid APIs, which introduces cost constraints,
(ii) the intention to minimize dependency on external services in order to maintain system robustness and architectural independence, and
(iii) the need to retain control over the quality and consistency of the data presented to the user, as directly integrating external API outputs may introduce unreliable or unsuitable information into the system.
Similarly, temporal constraints such as seasonal availability or event-based POIs are not modeled. While these features are valuable, they require continuous data updates and time-dependent logic that exceed the scope of the current system.

11.1.4 Dataset Coverage and Content Constraints

Although the dataset has been carefully constructed, its completeness is inherently limited by the availability and quality of external sources. Certain POIs—such as newly opened locations, small-scale attractions, or less-documented sites—may be missing or underrepresented.

In addition, multimedia content (images and audio) was curated and manually reviewed, but not evaluated through fully standardized or automated quality metrics. As a result, while content is generally correct and representative, variations in quality may still exist.

11.1.5 External Dependency Constraints

The system relies on external services for routing and certain data components. While integration correctness is ensured, the system does not independently verify the geographic accuracy or optimality of routes produced by the routing engine.
This introduces a dependency on external systems, which simplifies implementation but limits full control over routing behavior.

11.1.6 User Model and Interaction Constraints

GUIDE operates without persistent user accounts, and all interactions are session-based. This design choice was made deliberately to avoid collecting and storing user data, thereby simplifying privacy considerations and reducing system complexity related to data management. However, this also limits the system's ability to support long-term personalization, user-specific recommendations, and learning-based improvements over time.
In addition, implementing a full user management system (including authentication, profile handling, and administrative control mechanisms) would introduce significant development overhead that exceeds the scope and timeline of the project.
Similarly, user inputs are collected through structured forms rather than natural language interfaces. While this approach ensures precise control over constraints and improves system reliability, it limits flexibility and ease of use compared to more advanced interaction paradigms such as NLP-based input systems. This design choice was also influenced by development time constraints, as structured inputs allow for more controlled testing, validation, and verification processes within the scope of an academic project.

11.1.7 Development and Resource Constraints

The system was developed within the constraints of an academic timeline by a small team. Decisions regarding scope, feature set, and system complexity were therefore influenced by available time, development capacity, and resource limitations.
Many advanced features—such as real-time data integration, large-scale dataset expansion, and production-level infrastructure were consciously excluded to ensure that the core system could be completed, tested, and validated within the given timeframe.

11.2 Design Trade-offs

Several limitations of the system are a direct result of conscious design trade-offs rather than implementation gaps.
For instance, the use of heuristic methods instead of exact optimization is motivated by the need for real-time interaction. Similarly, the decision to operate on a curated dataset rather than real-time data sources ensures reliability and consistency.
These trade-offs reflect a design philosophy that prioritizes practical usability and system stability over theoretical optimality.

11.3 Relation to Future Work

Several of the limitations discussed above are closely related to the extensions outlined in the Future Work section. In particular, aspects such as real-time data integration, multi-modal transportation, global expansion, personalization, and dynamic dataset updates emerge as natural directions for extending the current system.
These features were considered during the design phase; however, they were intentionally excluded from the implementation, as they would require additional infrastructure, continuous data maintenance, and significantly increased system complexity beyond what could be realistically supported within the project scope and timeline.

11.4 Engineering Contributions

The GUIDE system demonstrates several key engineering contributions:

- Integration of data engineering, algorithmic design, and system architecture within a single platform.
- Development of a heuristic-based itinerary planning approach tailored for real-world constraints.
- Construction of a large-scale, curated POI dataset for Türkiye.
- Design of an interactive replanning mechanism enabling dynamic user control.

12 Conclusion and Future Work

This project presented GUIDE, an integrated system designed to generate structured, route-aware, multi-day travel itineraries based on user-defined constraints. The system combines dataset construction, heuristic itinerary generation, routing integration, and multimedia content delivery within a unified workflow.
The primary objective of the project was to reduce the fragmentation inherent in travel planning by providing a single platform that integrates POI discovery, route construction, and content exploration. Within the defined constraints of the project, the system has been successfully implemented and validated through comprehensive testing. The results indicate that GUIDE operates reliably, produces coherent itineraries, and satisfies its functional requirements.
At the same time, it is important to acknowledge that the system operates under practical limitations, particularly due to the heuristic nature of the planning algorithm, the reliance on curated datasets, and external service dependencies. Despite these constraints, the system demonstrates that a unified and efficient itinerary planning solution can be achieved within a manageable computational framework.

12.1 Future Work

The current system provides a strong foundation; however, several extensions can significantly improve both functionality and real-world applicability:
Future work directions can be grouped into several key areas:

12.1.1 Planning and Itinerary Enhancements

Integration of Travel Essentials: Extend the system to include restaurants, accommodation options, and other complementary travel components, enabling fully comprehensive trip planning.
Cost-Aware Planning: Incorporate pricing information such as entrance fees, transportation costs, and estimated travel expenses, allowing users to generate itineraries under budget constraints.
Temporal Constraints and Availability: Integrate opening hours, visiting days, and seasonal availability of POIs to ensure that generated itineraries are temporally feasible.
Real-Time Data Integration: Enhance the system with dynamic updates such as temporary closures, maintenance status, crowd levels, or real-time travel conditions.
Event-Based and Time-Dependent POIs: Extend the dataset to include time-specific activities such as concerts, exhibitions, and festivals, enabling itinerary generation that adapts to specific dates.
Multi-Modal Transportation Support: Extend the routing capabilities beyond car-based travel to include walking, public transportation, rail systems, and air travel. Integrating country-specific transportation infrastructures (e.g., metro, bus, train, and flight networks) would enable more realistic and flexible itinerary generation across different travel contexts.
Region-Level and Multi-City Itinerary Planning: Extend the system from single-city planning to region-based and multi-city itineraries (e.g., Black Sea tour). This would enable longer and more comprehensive travel plans that span multiple destinations, requiring adaptations in route generation, duration handling, and inter-

city transportation modeling.
Global Expansion and Localization: Extend the system beyond Türkiye to support international destinations. This would require adapting the dataset, category structures, and planning logic to different cultural contexts, as well as integrating country-specific data sources, languages, and travel infrastructures.

12.1.2 Data and Infrastructure Improvements

Dynamic Dataset Expansion: Introduce automated pipelines (e.g., web scraping or API integration) to continuously update the POI dataset with newly discovered locations.
User-Contributed Content: Develop mechanisms that allow users to suggest or contribute new POIs in a controlled and moderated manner.
Multimedia Quality Improvement: Improve the reliability and quality of images and audio content through more advanced filtering, validation, or multi-source aggregation.
Routing Engine Independence: Reduce dependency on external routing services by integrating or developing an internal routing solution, enabling greater system control and flexibility.
Scalability and Production Deployment: Adapt the system for real-world deployment by improving concurrency handling, infrastructure robustness, and large-scale user support.

12.1.3 User Experience and Interaction

Natural Language Input: Replace structured input forms with natural language-based interaction using NLP techniques, enabling more intuitive and flexible user input.
User Interface Enhancements: Further improve the frontend design to align with modern UI/UX standards and enhance usability, responsiveness, and visual appeal.
Mobile Application Deployment: Extend the system into a mobile application to improve accessibility and real-world usability during travel.
Offline Functionality: Provide offline access to itineraries, maps, and essential POI information, improving usability in low-connectivity environments during travel.
External Navigation Integration: Enable direct integration with navigation platforms such as Google Maps, allowing users to seamlessly export generated itineraries and follow routes using familiar navigation tools.

12.1.4 Personalization and Social Features

Personalization and User Modeling: Introduce user accounts and history tracking to enable personalized itinerary generation based on previous interactions and preferences.
Personal Travel Logging: Allow users to attach personal notes, photos, and experiences to visited POIs, enabling the system to function as a lightweight travel journal alongside itinerary planning.
Social Features: Allow users to share itineraries, explore routes created by others, and integrate social interaction mechanisms inspired by modern travel platforms.
Finally, the GUIDE project is designed as an extensible and evolving system. The complete implementation and dataset are shared as an open-source repository to support further development and community contributions:
`https://github.com/tunakodal/quad-core.git`

Note: Spell checking and basic grammatical verification were performed using Grammarly: (`https://www.grammarly.com`).

References

[1] Amazon s3: Scalable cloud storage. `https://aws.amazon.com/s3/`. Accessed: 2026.

[2] Google places api. `https://developers.google.com/maps/documentation/places`. Accessed: 24 March 2026.

[3] Republic of türkiye ministry of culture and tourism. `https://ktb.gov.tr`. Accessed: 2026.

[4] Leaflet.js: An open-source javascript library for interactive maps. `https://leafletjs.com/`. Accessed: 2026.

[5] Türkiye museums official website. `https://muze.gov.tr`. Accessed: 2026.

[6] Node.js. https://nodejs.org/.

[7] Openstreetmap. https://www.openstreetmap.org. Accessed: 24 March 2026.

[8] React. https://react.dev/.

[9] Republic of türkiye ministry of agriculture and forestry. https://tarimorman.gov.tr. Accessed: 2026.

[10] Wikidata. https://www.wikidata.org, . Accessed: 24 March 2026.

[11] Wikipedia. https://www.wikipedia.org, . Accessed: 24 March 2026.

[12] Anthropic. Claude haiku, 2025. URL https://www.anthropic.com/claude. Accessed March 20, 2026.

[13] T. Bray. The javascript object notation (json) data interchange format. https://www.rfc-editor.org/rfc/rfc8259, 2017. RFC 8259.

[14] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33: 1877–1901, 2020.

[15] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1): 1–43, 2012. doi: 10.1109/TCIAIG.2012.2186810.

[16] European Union. General data protection regulation (gdpr). https://eur-lex.europa.eu/eli/reg/2016/679/oj, 2016. Accessed: 2026-03-30.

[17] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. URL https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[18] Damianos Gavalas, Vlasios Kasapakis, Charalampos Konstantopoulos, Grammati Pantziou, and Nikolaos Vathis. Mobile recommender systems in tourism. *Journal of Network and Computer Applications*, 39: 319–333, 2014. doi: 10.1016/j.jnca.2013.04.006.

[19] Damianos Gavalas, Charalampos Konstantopoulos, Konstantinos Mastakas, and Grammati Pantziou. A survey on algorithmic approaches for solving tourist trip design problems. *Journal of Heuristics*, 20(3): 291–328, 2014. doi: 10.1007/s10732-014-9242-5.

[20] Git Project. Git. https://git-scm.com/, 2024. Accessed: 2026-03-29.

[21] GitHub Inc. Github. https://github.com/, 2024. Accessed: 2026-03-29.

[22] Google. Google maps. https://maps.google.com, 2026. Retrieved March 23, 2026.

[23] Google Cloud. Cloud text-to-speech api. https://cloud.google.com/text-to-speech, 2026. Accessed: 2026.

[24] GraphHopper. Graphhopper routing and optimization platform. https://www.graphhopper.com/products/, 2026. Retrieved March 23, 2026.

[25] GUIDE Project Team. Guide project: Analysis report. https://prism.openai.com/?u=cd302634-f3dc-43ab-8999-2a1979115d13&pg=1&m=main.tex&d=7, 2025. Retrieved March 28, 2026.

[26] GUIDE Project Team. Guide project: High-level design report. https://prism.openai.com/?u=6ae9e170-5330-4dec-a098-ecd39a396b2a&pg=1&m=main.tex&d=7, 2025. Retrieved March 28, 2026.

[27] GUIDE Project Team. Guide project: Project proposal report. https://prism.openai.com/?u=d73f7893-3f69-4f01-993e-6cf2c81fde60&pg=1&m=main.tex&d=7, 2025. Retrieved March 28, 2026.

[28] GUIDE Project Team. Guide project: Project specification report. https://prism.openai.com/?u=bfdb1182-e9a2-4a67-9eb1-1652d98e749e&pg=1&m=main.tex&d=7, 2025. Retrieved March 28, 2026.

[29] GUIDE Project Team. Guide project: Low-level design document. https://prism.openai.com/?u=bf41c171-acbc-4c02-93d3-288e1200907b&pg=1&m=main.tex&d=7, 2026. Retrieved March 28, 2026.

[30] GUIDE Project Team. Guide project: Test plan report. https://prism.openai.com/?u=93126d1e-e18b-46ea-93f8-de1d5ba735e8&pg=1&m=main.tex&d=7, 2026. Retrieved March 28, 2026.

[31] JetBrains s.r.o. Pycharm integrated development environment. https://www.jetbrains.com/pycharm/, 2024. Accessed: 2026-03-29.

[32] Eugene L. Lawler et al. The traveling salesman problem: A case study. *Combinatorial Optimization*, 1985.

[33] Layla. Layla ai travel planner. https://layla.ai/, 2026. Retrieved March 23, 2026.

[34] Open Source Routing Machine. Open source routing machine (osrm). https://project-osrm.org, 2026. Retrieved March 23, 2026.

[35] OpenTripPlanner. Opentripplanner documentation. https://docs.opentripplanner.org/en/latest/, 2026. Retrieved March 23, 2026.

[36] PostgreSQL Global Development Group. Postgresql. https://www.postgresql.org/, 2024. Accessed: 2026-03-29.

[37] Python Software Foundation. Python programming language. https://www.python.org/, 2024. Accessed: 2026-03-29.

[38] Republic of Türkiye. Law on the protection of personal data no. 6698 (kvkk). https://www.kvkk.gov.tr/, 2016. Accessed: 2026-03-30.

[39] Roadtrippers. Roadtrippers. https://roadtrippers.com, 2026. Retrieved March 23, 2026.

[40] Supabase Inc. Supabase: The open source firebase alternative. https://supabase.com/, 2024. Accessed: 2026-03-29.

[41] Sygic. Sygic travel maps. https://www.sygic.com/travel, 2026. Retrieved March 23, 2026.

[42] TripAdvisor. Tripadvisor trips. https://www.tripadvisor.com/Trips, 2026. Retrieved March 23, 2026.

[43] Pieter Vansteenwegen, Wouter Souffriau, and Dirk Van Oudheusden. The orienteering problem: A survey. *European Journal of Operational Research*, 209 (1):1–10, 2011. doi: 10.1016/j.ejor.2010.03.045.

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[45] Wanderlog. Wanderlog travel planner. https://wanderlog.com, 2026. Retrieved March 23, 2026.

13 Appendices

A Example POI Record

A representative POI record is presented below in JSON format to illustrate the structure of the final dataset. The example demonstrates how geographical attributes, hierarchical categories, multilingual descriptions, and multimedia references are organized within a single record in the GUIDE data model.


```
{
  "name": "Hagia Sophia",
  "lat": 41.00833333,
  "lon": 28.98,
  "categories": ["Museum", "Mosque", "Church", "Complex"],
  "main_1": "Cultural Heritage",
  "main_2": "Museums",
  "sub_1": "Religious",
  "sub_2": "Museum",
  "sub_3": "Urban & Monumental Heritage",
  "sub_4": null,
  "city": "Istanbul",
  "medium_description_tr": "Ayasofya, İstanbul'un en ikonik yapılarından biri olup, 532-537 yılları arasında Bizans İmparatoru I. Justiniyen tarafından inşa ettirilmiştir. Dünya mimarisinin başyapıtı sayılan bu yapı, görkemli kubbesiyle ve içindeki zarif mozaiklerle ziyaretçilerini büyüler. 1453'te İstanbul'un fethedilmesinden sonra camiye dönüştürülen Ayasofya, Osmanlı döneminde de önemli bir dini merkez olarak işlev görmüştür. Müze olarak uzun yıllar hizmet veren Ayasofya, Sultanahmet'in merkezinde yer alır ve hem mimari hem kültürel değeriyle UNESCO Dünya Mirası Listesi'nde yer almaktadır.",
  "medium_description_de": "Die Hagia Sophia steht als eines der ikonischsten Bauwerke Istanbuls, das der byzantinische Kaiser Justinian I. zwischen 532 und 537 n. Chr. in Auftrag gab und das als Meisterwerk der Weltarchitektur gilt. Ihre prächtige Kuppel und die feinen Innenmosaiken faszinieren bis heute Besucher aus aller Welt mit ihrer atemberaubenden Schönheit. Nach der Eroberung Konstantinopels durch die Osmanen 1453 wurde dieses bemerkenswerte Gebäude in eine Moschee umgewandelt und diente während der osmanischen Zeit als wichtiges Religionszentrum. Im Herzen von Sultanahmet gelegen, ist die Hagia Sophia als UNESCO-Weltkulturerbe anerkannt und bleibt ein Zeugnis der byzantinischen und osmanischen Architektur und Kulturleistungen.",
  "medium_description_en": "Hagia Sophia stands as one of Istanbul's most iconic structures, commissioned by Byzantine Emperor Justinian I between 532-537 CE and considered a masterpiece of world architecture. Its magnificent dome and delicate interior mosaics continue to captivate visitors from around the world with their breathtaking beauty. After the Ottoman conquest of Constantinople in 1453, this remarkable building was converted into a mosque and served as an important religious center throughout the Ottoman period. Located in the heart of Sultanahmet, Hagia Sophia is recognized as a UNESCO World Heritage Site and remains a testament to both Byzantine and Ottoman architectural and cultural achievements.",
  "google_place_id": "ChIJJxwBkr65yhQRrk9EN29vbiM",
  "google_rating": 4.8,
  "google_reviews_total": 134203,
```

```
  "audio_tr": "https://guide-media2.s3.eu-north-1.amazonaws.com/audio/Istanbul_Anatolia/Hagia_Sophia/Hagia_Sophia_tr.mp3",
  "audio_de": "https://guide-media2.s3.eu-north-1.amazonaws.com/audio/Istanbul_Anatolia/Hagia_Sophia/Hagia_Sophia_de.mp3",
  "audio_en": "https://guide-media2.s3.eu-north-1.amazonaws.com/audio/Istanbul_Anatolia/Hagia_Sophia/Hagia_Sophia_en.mp3",
  "photo_urls": [
    "https://guide-media2.s3.eu-north-1.amazonaws.com/photos/Istanbul_(Anatolia)/Hagia_Sophia/Hagia_Sophia_1.jpg",
    "https://guide-media2.s3.eu-north-1.amazonaws.com/photos/Istanbul_(Anatolia)/Hagia_Sophia/Hagia_Sophia_2.jpg",
    "https://guide-media2.s3.eu-north-1.amazonaws.com/photos/Istanbul_(Anatolia)/Hagia_Sophia/Hagia_Sophia_3.jpg"
  ],
  "photo_files": [
    "poi_photos\\Istanbul_Anatolia\\Hagia_Sophia_1.jpg",
    "poi_photos\\Istanbul_Anatolia\\Hagia_Sophia_2.jpg",
    "poi_photos\\Istanbul_Anatolia\\Hagia_Sophia_3.jpg"
  ],
  "photo_count": 3
}
```


B Category-Based Visit Duration Values

Subcategory	Base Duration (min)
Ancient & Archaeology	75
Museum	75
Fortifications	60
Civil & Traditional Architecture	50
Terrain & Landforms	50
Wildlife & Natural Experience	50
Parks & Outdoor	40
Water & Coastal	40
Urban & Monumental Heritage	40
Transportation as Heritage	40
Historical Infrastructure	40
Religious	30

Table 8: Category-based base visit duration values used in the estimation model.

C Itinerary Generation Algorithm

Algorithm 1 Monte Carlo-based Itinerary Generation

```
Retrieve POIs based on selected city and categories
Initialize POI pool P
for day = 1 to D do
    best_route ← ∅
    fallback ← ∅
    for attempt = 1 to 3 do
        Generate N = 500 candidate routes from P
    (each containing between 4 and 9 POIs per day)
        for each candidate route r do
            Determine POI order via nearest-neighbor heuristic
            Compute total route distance with 1.65× Haversine
            if d(r) > 2 · d_target then
                score(r) ← 0
            else
                Compute composite score(r)
            end if
        end for
        Sort candidates by score in descending order
        if fallback = ∅ then
            fallback ← highest-scoring candidate
        end if
        Select top-10 candidates
        for each candidate r in top-10 do
            if score(r) ≤ 0 then
                continue
            end if
            duration(r) ← Σ_{p∈r} visit_duration(p)
            if duration(r) ≤ 420 and 0.75 d_target ≤ d(r) ≤ 1.25 d_target then
                best_route ← r
                break
            end if
        end for
        if best_route ≠ ∅ then
            break
        end if
    end for
    if best_route = ∅ then
        best_route ← fallback
    end if
    Add best_route to itinerary
    Remove POIs in best_route from P
end for
return final itinerary
```

D Frontend Structure

The frontend implementation is organized into pages, layout components, UI elements, and supporting modules for API communication and map-based visualization. The overall structure is shown below:

```
frontend/
 App.jsx
 App.module.css
 api/
    client.ts
    types.ts
 layout/
    AppShell.jsx
    Footer.jsx
    Header.jsx
 lib/
    leafletFix.jsx
 main.jsx
 pages/
    HowItWorks.jsx
    Journey.jsx
    Landing.jsx
    Planning.jsx
    Poi.jsx
    Replanning.jsx
    Route.jsx
    RouteMap.jsx
 styles/
    AppShell.module.css
    Footer.module.css
    Header.module.css
    HowItWorks.module.css
    Journey.module.css
    Landing.module.css
    Planning.module.css
    Poi.module.css
    Replanning.module.css
    Route.module.css
    globals.css
    theme.css
 ui/
     Button.jsx
     Button.module.css
```

E Backend Structure

The backend implementation follows a modular structure aligned with the Low-Level Design. The overall module and directory organization is shown below.

```
backend/
 app/
    api/
       error_mapper.py
       validator.py
       routes/
           poi_endpoints.py
           route_endpoints.py
    core/
       config.py
       containers.py
       database.py
    integration/
       osrm_client.py
    models/
       domain.py
       enums.py
       geo.py
       media.py
       poi.py
       route.py
    repositories/
       content_repository.py
       interfaces.py
       media_repository.py
       poi_repository.py
       repositories.py
       stubs.py
    schemas/
       common.py
       dtos.py
       poi_dtos.py
       route_dtos.py
       suggestion_dtos.py
       travel.py
    services/
        content_service.py
        itinerary_builder.py
        itinerary_planner.py
        itinerary_service.py
        plan_ranker.py
        poi_service.py
        routing_service.py
 data/
    contents.json
    pois.json
 db/
    schema.sql
 tests/
    unit/
        test_poi_service.py
        test_validator.py
 main.py
 requirements.txt
 pytest.ini
```